

Machine Learning Laboratory Manual

Completed Edition with Full Implementations and Results

Practical Guide to Classical, Ensemble, Neural, Probabilistic,
Density-Based, Semi-Supervised and Generative ML

Prepared by

Dr. Ohidujjaman Tuhin

Associate Professor, Dept. of CSE, UIU

Department of Computer Science & Engineering
Machine Learning Lab

September 25, 2026

Contents

Preface	5
Experiment 1: K-means Clustering	6
1. Objective	6
2. Dataset Used	6
3. Required Libraries	6
4. Brief Theory	6
5. Procedure	6
6. Reference Implementation	6
7. Results	7
8. Discussion and Conclusion	7
9. Answers to Viva Questions	7
10. Complete Code Listing	8
Experiment 3: Hierarchical Clustering	10
1. Objective	10
2. Dataset Used	10
3. Required Libraries	10
4. Brief Theory	10
5. Procedure	10
6. Reference Implementation	10
7. Results	10
8. Discussion and Conclusion	10
9. Answers to Viva Questions	11
10. Complete Code Listing	11
Experiment 4: Fuzzy C-means	13
1. Objective	13
2. Dataset Used	13
3. Required Libraries	13
4. Brief Theory	13
5. Procedure	13
6. Reference Implementation	13
7. Results	13
8. Discussion and Conclusion	14
9. Answers to Viva Questions	14
10. Complete Code Listing	14
Experiment 5: DBSCAN	17
1. Objective	17
2. Dataset Used	17
3. Required Libraries	17
4. Brief Theory	17

5. Procedure	17
6. Reference Implementation	17
7. Results	17
8. Discussion and Conclusion	18
9. Answers to Viva Questions	18
10. Complete Code Listing	18
Experiment 6: HDBSCAN	21
1. Objective	21
2. Dataset Used	21
3. Required Libraries	21
4. Brief Theory	21
5. Procedure	21
6. Reference Implementation	21
7. Results	21
8. Discussion and Conclusion	22
9. Answers to Viva Questions	22
10. Complete Code Listing	22
Experiment 7: Self-training for Semi-supervised Learning	24
1. Objective	24
2. Dataset Used	24
3. Required Libraries	24
4. Brief Theory	24
5. Procedure	24
6. Reference Implementation	24
7. Results	24
8. Discussion and Conclusion	25
9. Answers to Viva Questions	25
10. Complete Code Listing	25
Experiment 8: Random Forest Regression (RFR)	28
1. Objective	28
2. Dataset Used	28
3. Required Libraries	28
4. Brief Theory	28
5. Procedure	28
6. Reference Implementation	28
7. Results	28
8. Discussion and Conclusion	28
9. Answers to Viva Questions	29
10. Complete Code Listing	29
Experiment 9: Random Forest Classification (RFC)	31
1. Objective	31
2. Dataset Used	31

3. Required Libraries	31
4. Brief Theory	31
5. Procedure	31
6. Reference Implementation	31
7. Results	31
8. Discussion and Conclusion	32
9. Answers to Viva Questions	32
10. Complete Code Listing	32
Experiment 10: XGBoost	34
1. Objective	34
2. Dataset Used	34
3. Required Libraries	34
4. Brief Theory	34
5. Procedure	34
6. Reference Implementation	34
7. Results	34
8. Discussion and Conclusion	34
9. Answers to Viva Questions	35
10. Complete Code Listing	35
Experiment 11: AdaBoost	37
1. Objective	37
2. Dataset Used	37
3. Required Libraries	37
4. Brief Theory	37
5. Procedure	37
6. Reference Implementation	37
7. Results	37
8. Discussion and Conclusion	37
9. Answers to Viva Questions	38
10. Complete Code Listing	38
Experiment 12: CatBoost	40
1. Objective	40
2. Dataset Used	40
3. Required Libraries	40
4. Brief Theory	40
5. Procedure	40
6. Reference Implementation	40
7. Results	40
8. Discussion and Conclusion	41
9. Answers to Viva Questions	41
10. Complete Code Listing	41
Experiment 13: Multilayer Perceptron (MLP)	43

1. Objective	43
2. Dataset Used	43
3. Required Libraries	43
4. Brief Theory	43
5. Procedure	43
6. Reference Implementation	43
Experiment 14: Recurrent Neural Network (RNN)	44
1. Objective	44
2. Datasets Used	44
3. Required Libraries	44
4. Brief Theory	44
5. Procedure	44
6. Reference Implementation	44
Experiment 15: Self-Organizing Map (SOM)	45
1. Objective	45
2. Dataset Used	45
3. Required Libraries	45
4. Brief Theory	45
5. Procedure	45
6. Reference Implementation	45
7. Results	45
8. Discussion and Conclusion	46
9. Answers to Viva Questions	46
10. Complete Code Listing	46
Experiment 16: Hidden Markov Model (HMM)	48
1. Objective	48
2. Dataset Used	48
3. Required Libraries	48
4. Brief Theory	48
5. Procedure	48
6. Reference Implementation	48
7. Results	48
8. Discussion and Conclusion	49
9. Answers to Viva Questions	49
10. Complete Code Listing	49
Experiment 17: Support Vector Machine (SVM)	52
1. Objective	52
2. Datasets Used	52
3. Required Libraries	52
4. Brief Theory	52
5. Procedure	52
6. Reference Implementation	52

7. Results	52
8. Discussion and Conclusion	53
9. Answers to Viva Questions	53
10. Complete Code Listing	53
Experiment 18: Large Language Model (LLM) Experiment	56
1. Objective	56
2. Dataset Used	56
3. Required Libraries	56
4. Brief Theory	56
5. Procedure	56
6. Reference Implementation	56
7. Results	56
8. Discussion and Conclusion	57
9. Answers to Viva Questions	57
10. Complete Code Listing	58
Experiment 19: Generalized Regression Neural Network (GRNN)	62
1. Objective	62
2. Dataset Used	62
3. Required Libraries	62
4. Brief Theory	62
5. Procedure	62
6. Reference Implementation	62
7. Results	62
8. Discussion and Conclusion	63
9. Answers to Viva Questions	63
10. Complete Code Listing	63
Integrated Mini Project: Comparative Machine Learning Study	66
Classification comparison — Heart Disease (same stratified 75/25 split, seed 42)	66
Regression comparison — California Housing (5,000 sample, same 80/20 split)	67
Clustering comparison — Mall Customers (K=5, standardized)	67
Density-based comparison — 7,638 real earthquakes ($M \geq 4.5$, 2023)	68
Appendix A: Quick Reference Formulas	70
Appendix B: Reproducibility Checklist	71
Appendix C: Suggested Assessment Rubric	72
Appendix D: Student Experiment Record	73

Preface

This manual covers the complete Machine Learning Laboratory sequence: clustering (K-means, modified K-means, hierarchical, fuzzy C-means), density-based learning (DBSCAN, HDBSCAN), semi-supervised learning (self-training), ensemble learning (random forest regression/classification, XGBoost, AdaBoost, CatBoost), neural networks (MLP, RNN/LSTM), self-organizing maps, hidden Markov models, support vector machines, large language models and generalized regression neural networks.

Every experiment follows the same eight-part report format: Objective; Suggested Dataset; Required Libraries; Brief Theory; Procedure; Reference Implementation; Evaluation / Expected Output; and Viva / Discussion Questions, extended with the complete executed code listing and the observed results. All experiments were run on real, public datasets with fixed random seeds; the full programs are included so that each experiment can be reproduced standalone.

Environment. Python 3.12, NumPy/Pandas, scikit-learn, XGBoost, CatBoost, PyTorch (CUDA), Hugging Face transformers, MiniSom and hmmlearn. Random seeds are fixed throughout (`random_state=42`).

Experiment 1: K-means Clustering

Category: Clustering

1. Objective

Partition unlabeled observations into K clusters by iteratively assigning points to the nearest centroid and updating centroids; select K using the elbow and silhouette diagnostics.

2. Dataset Used

Mall Customers (data/mall_customers.csv), 200 customers, two features: Annual Income (k\$) and Spending Score (1-100). Features are standardized (StandardScaler).

3. Required Libraries

numpy, pandas, matplotlib, scikit-learn

4. Brief Theory

K-means minimizes the within-cluster sum of squares $J = \sum_k \sum_{x \in C_k} ||x - \mu_k||^2$. It alternates assignment and centroid update until convergence (a local optimum). Euclidean distance is scale-sensitive, so features are standardized. K is chosen externally, e.g. by the elbow (inertia) and silhouette (cohesion vs separation).

5. Procedure

1. Load the dataset and keep the two numerical features.
2. Standardize the features.
3. Try $K = 2 \dots 10$; record inertia (WCSS) and silhouette for each K.
4. Choose K at the elbow / silhouette peak and fit the final model with `init='k-means++', n_init=20, random_state=42`.
5. Visualize clusters and centroids.
6. Report silhouette, Davies-Bouldin and Calinski-Harabasz.

6. Reference Implementation

```
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    kmeans.fit(X_mall_scaled)
    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_))
## 10. Complete Code Listing

*Full runnable program for this experiment, extracted from `notebooks/01_clustering_algorithms.ipynb`
↳ (executed; results above are its actual output).*

**Listing 1.1 – Core numerical and plotting libraries**

```python

Experiment 2: Modified K-means

Category: Clustering

1. Objective
Implement the prescribed transparent modification of K-means: K-Means++ initialization plus an
↳ explicit outlier-distance check with a documented percentile threshold before final cluster
```

↪ interpretation.

## ## 2. Dataset Used

Mall Customers (200 × 2, standardized). The modification **is** applied on top of the same features.

## ## 3. Required Libraries

numpy, matplotlib, scikit-learn

## ## 4. Brief Theory

"Modified K-means" **is not** a single standard algorithm; the manual defines it precisely **for**

- ↪ reproducibility: initialize **with** K-Means++, fit K-means, compute each sample's distance to its
- ↪ assigned centroid, take a percentile threshold (97th percentile) and flag more distant samples
- ↪ as candidate outliers. It is an experimental variant that adds outlier awareness, not a
- ↪ replacement for robust clustering.

## ## 5. Procedure

1. Fit K-means with ``init='k-means++'``, `K = 5`.
2. Compute `d_i = ||x_i - μ_{c_i}||` for every sample.
3. Set threshold = 97th percentile of the training distances.
4. Flag samples with `d_i > threshold` as candidate outliers.
5. Plot centroids/clusters with flagged points highlighted.
6. Compare with the standard labels.

## ## 6. Reference Implementation

```
```python
km_mod = KMeans(n_clusters=5, init='k-means++', n_init=20, random_state=42)
mod_labels = km_mod.fit_predict(X_mall_scaled)
dist_to_centroid = np.linalg.norm(X_mall_scaled - km_mod.cluster_centers_[mod_labels], axis=1)
threshold = np.percentile(dist_to_centroid, 97)
outlier_flag = dist_to_centroid > threshold
```

7. Results

Item	Value
Distance threshold (97th percentile)	1.072 (standardized units)
Flagged candidate outliers	6 of 200 (3%)
Cluster labels	identical to standard K-means (same partition)

The flagged customers are the most weakly attached points of their clusters — typically boundary customers with unusual income/spending combinations. The cluster structure itself is unchanged, which is expected: the modification adds an outlier interpretation layer.

8. Discussion and Conclusion

The variant is useful when the analyst needs to know which observations are poorly represented by any centroid (e.g., unusual customers). Because the threshold is a documented percentile, the flag rate is controlled and reproducible (3% here). The limitation is that distance-to-centroid is biased for elongated or density-varying clusters, and the flagged points are not automatically errors.

9. Answers to Viva Questions

- **Why is it important to define a modification precisely?** Without a formal definition, “modified K-means” is ambiguous; a precise version (init method, threshold rule, percentile) is reproducible and testable.
- **How does K-Means++ differ from random initialization?** K-Means++ selects initial centroids with probability proportional to squared distance to the nearest chosen centroid, spreading seeds out and reducing convergence to poor local optima.
- **Why might distance-to-centroid fail for elongated clusters?** Centroid distance is

isotropic; a point can be close to the centroid along the long axis yet far along the short axis, so elongated clusters can flag legitimate points and missing real outliers.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/01_clustering_algorithms`.
 ↪ `ipynb` (executed; results above are its actual output).

Listing 2.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
  ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Listing 2.2 — Load the Mall Customers dataset

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 2.3 — Modified K-Means (manual variant): K-Means++ + outlier-distance check

```
# ---- Modified K-Means (manual variant): K-Means++ + outlier-distance check ----
km_mod = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
mod_labels = km_mod.fit_predict(X_mall_scaled)

# Distance of every sample to its assigned centroid
dist_to_centroid = np.linalg.norm(X_mall_scaled - km_mod.cluster_centers_[mod_labels], axis=1)
threshold = np.percentile(dist_to_centroid, 97) # documented percentile
outlier_flag = dist_to_centroid > threshold

print(f"Distance threshold (97th percentile): {threshold:.3f}")
```

```
print(f"Flagged candidate outliers: {outlier_flag.sum()} of {len(X_mall)}")

# ---- Plot clusters with flagged candidate outliers highlighted ----
plt.figure(figsize=(9, 6))
for cluster_id in range(optimal_k):
    pts = X_mall[mod_labels == cluster_id]
    plt.scatter(pts[:, 0], pts[:, 1], s=45, edgecolors='k', alpha=0.8)
plt.scatter(X_mall[outlier_flag, 0], X_mall[outlier_flag, 1], s=180, facecolors='none',
            edgecolors='red', linewidths=2.5, label=f'Flagged outliers ({outlier_flag.sum()})')
plt.title("Modified K-Means: K-Means++ with Outlier-Distance Flagging", fontsize=13, fontweight='bold',
        ↪ ')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```

Experiment 3: Hierarchical Clustering

Category: Clustering

1. Objective

Perform agglomerative hierarchical clustering, interpret the dendrogram, and compare Ward linkage with the other algorithms.

2. Dataset Used

Mall Customers (200 × 2, standardized).

3. Required Libraries

scipy, scikit-learn, matplotlib

4. Brief Theory

Agglomerative clustering starts with one cluster per point and repeatedly merges the closest pair. Ward linkage merges the pair that increases total within-cluster variance the least: $\Delta W(A,B) = (n_A n_B)/(n_A+n_B) ||\mu_A - \mu_B||^2$. The dendrogram shows all merges; a horizontal cut determines the number of clusters.

5. Procedure

1. Standardize the features.
2. Compute the Ward linkage matrix.
3. Plot the dendrogram (truncated for readability) and choose a cut level.
4. Fit `AgglomerativeClustering(n_clusters=5, linkage='ward')`.
5. Evaluate with silhouette, Davies-Bouldin and Calinski-Harabasz.
6. Discuss linkage choice.

6. Reference Implementation

```
linkage_matrix = linkage(X_mall_scaled, method='ward')
dendrogram(linkage_matrix, truncate_mode='lastp', p=25, leaf_rotation=45, show_contracted=True)
agg_model = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_mall_scaled)
```

7. Results

Metric	Value
Silhouette Score	0.5538
Davies-Bouldin Index	0.5779
Calinski-Harabasz Score	244.41

The dendrogram shows a clear large merge gap that supports cutting at five clusters. The partition is nearly identical in quality to K-means (silhouette difference < 0.001).

8. Discussion and Conclusion

Ward linkage produces compact, balanced clusters and requires no initialization, at the cost of $O(n^2)$ memory and the need to choose a cut. Single linkage would chain across the space and complete linkage would fragment; Ward is the appropriate default for compact clusters of

similar size.

9. Answers to Viva Questions

- **What does a dendrogram show?** The full merge hierarchy: leaf order, merge heights (distances) and the structure of nested clusters; cutting at a height yields a flat partition.
- **What is linkage?** The rule that defines the distance between two clusters (single = closest pair, complete = farthest pair, average = mean pair, Ward = variance increase).
- **When is Ward linkage inappropriate?** With non-spherical/elongated clusters, unequal cluster sizes or outliers — it assumes compact, similar-variance clusters.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/01_clustering_algorithms.ipynb` (executed; results above are its actual output).

Listing 3.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
↳ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Listing 3.2 — Load the Mall Customers dataset

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 3.3 — Build the linkage matrix and draw the dendrogram

```
# ---- Build the linkage matrix and draw the dendrogram ----
# 'ward' linkage merges the pair of clusters that increases within-cluster variance the least.
```

```
linkage_matrix = linkage(X_mall_scaled, method='ward')

# The dendrogram shows every merge; we truncate it to the last 25 merges for readability.
plt.figure(figsize=(12, 5))
dendrogram(
    linkage_matrix,
    truncate_mode='lastp',  # show only the last p merged clusters
    p=25,
    leaf_rotation=45,
    leaf_font_size=10,
    show_contracted=True    # condensed leaves are shown as counts
)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)", fontsize=14, fontweight='bold')
plt.xlabel("Cluster Sample Index / Merged Size", fontsize=12)
plt.ylabel("Euclidean Distance (Ward Threshold)", fontsize=12)
plt.axhline(y=10.0, color='r', linestyle='--', label='Cut-off Threshold (K=5)') # horizontal cut =>
    ↪ cluster count
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 3.4 — Fit the final Agglomerative model (Ward linkage, K = 5)

```
# ---- Fit the final Agglomerative model (Ward linkage, K = 5) ----
agg_model = AgglomerativeClustering(n_clusters=optimal_k, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_mall_scaled)

print(f"Hierarchical Silhouette Score: {silhouette_score(X_mall_scaled, agg_labels):.4f}")
print(f"Hierarchical Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, agg_labels):.4f}")
```

Experiment 4: Fuzzy C-means

Category: Clustering

1. Objective

Cluster data with soft memberships so each observation can belong to multiple clusters with different degrees; report the fuzzy partition coefficient (FPC).

2. Dataset Used

Mall Customers (200 × 2, standardized), K = 5, fuzzifier m = 2.0.

3. Required Libraries

numpy, pandas, matplotlib (the c-means equations are implemented from scratch; scikit-fuzzy is an equivalent library implementation)

4. Brief Theory

Fuzzy C-means minimizes $J_m = \sum_i \sum_k u_{ik}^m ||x_i - v_k||^2$ with memberships $u_{ik} \in [0,1]$, $\sum_k u_{ik} = 1$. It alternates: $v_k = \sum_i u_{ik}^m x_i / \sum_i u_{ik}^m$ and $u_{ik} = 1 / \sum_j (d_{ik}/d_{ij})^{2/(m-1)}$. The fuzzifier m controls softness (m → 1 becomes K-means). FPC = $(1/N) \sum_i \sum_k u_{ik}^2$ measures fuzziness (1 = hard, 1/K = maximally fuzzy).

5. Procedure

1. Standardize the data.
2. Initialize the membership matrix randomly (rows sum to 1).
3. Alternate centroid and membership updates until $\max|\Delta U| < 1e-5$.
4. Derive hard labels via argmax membership.
5. Report metrics and FPC; visualize memberships/hard partition.
6. Discuss ambiguous samples and m.

6. Reference Implementation

```
Um = U ** self.m
centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]
dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2) + 1e-10
U = (1.0 / dist) ** (2.0 / (self.m - 1.0))
U = U / U.sum(axis=1, keepdims=True)
labels = np.argmax(U, axis=1)
```

7. Results

Metric	Value
Silhouette Score (hard labels)	0.5547
Davies-Bouldin Index	0.5722
Fuzzy Partition Coefficient (FPC)	0.6711
Iterations to convergence	< 50

FPC = 0.67 (between $1/K = 0.2$ and 1.0) shows the partition is mostly crisp but retains soft boundary memberships. Customers with membership vectors close to (0.5, 0.5, ...) sit between clusters and are the genuinely ambiguous ones.

8. Discussion and Conclusion

FCM converges to the same hard partition as K-means here (silhouette 0.5547) because the Mall Customers clusters are well separated; the added value is the membership matrix, which quantifies uncertainty at boundaries. Larger m makes memberships softer (lower FPC) without changing the hard structure much; smaller m (~ 1.1) approaches K-means.

9. Answers to Viva Questions

- **How is Fuzzy C-means different from K-means?** It assigns membership degrees instead of hard labels, so each point belongs to all clusters with weights that sum to 1; K-means is the limiting hard case.
- **What does m control?** The fuzziness exponent: larger m spreads membership across clusters (softer), m approaching 1 gives crisp K-means-like assignments.
- **What does a membership vector represent?** The degree to which one observation belongs to each cluster; near-uniform values indicate an ambiguous point on a cluster boundary.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/01_clustering_algorithms`.
 ↪ `ipynb` (executed; results above are its actual output).

Listing 4.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
  ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Listing 4.2 — Load the Mall Customers dataset

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values
```

```
# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 4.3 — Fuzzy C-Means class (from scratch)

```
class FuzzyCMeans:
    """
    Fuzzy C-Means (FCM) clustering - implemented from scratch with NumPy.

    Each sample belongs to every cluster with a membership degree  $u_{ik}$  in  $[0, 1]$ ,
    and the memberships over the  $K$  clusters sum to 1 per sample.

    Parameters
    -----
    n_clusters    : number of fuzzy clusters  $K$ 
    m             : fuzzifier exponent ( $m > 1$ ; larger  $m \Rightarrow$  fuzzier memberships)
    max_iter      : maximum number of alternating-optimization iterations
    tol           : convergence tolerance on the maximum membership change
    random_state  : seed for reproducible random initialization
    """

    def __init__(self, n_clusters=5, m=2.0, max_iter=150, tol=1e-5, random_state=42):
        self.n_clusters = n_clusters
        self.m = m
        self.max_iter = max_iter
        self.tol = tol
        self.random_state = random_state

    def fit(self, X):
        rng = np.random.RandomState(self.random_state)
        n_samples = X.shape[0]

        # Step 1: initialize the membership matrix  $U$  randomly and normalize every row to sum to 1
        U = rng.rand(n_samples, self.n_clusters)
        U = U / U.sum(axis=1, keepdims=True)

        # Step 2: alternate between centroid and membership updates until convergence
        for iteration in range(self.max_iter):
            U_old = U.copy()

            # 2a: centroid update - membership-weighted mean of the data
            Um = U ** self.m                                #  $u_{ik}^m$ 
            centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]

            # 2b: compute the ( $n_{\text{samples}} \times n_{\text{clusters}}$ ) distance matrix to every centroid
            dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2)
            dist = np.fmax(dist, 1e-10) # avoid division by zero

            # 2c: membership update - closer points receive higher membership
            #  $u_{ik} = 1 / \sum_j (d_{ik} / d_{ij})^{2/(m-1)}$ 
            inv_dist = 1.0 / dist
            power = 2.0 / (self.m - 1.0)
            inv_dist_p = inv_dist ** power
            U = inv_dist_p / inv_dist_p.sum(axis=1, keepdims=True)

            # 2d: stop when memberships stop changing (convergence)
            if np.max(np.abs(U - U_old)) < self.tol:
                break

        # Store results: final centroids, full membership matrix, and hard labels
        self.cluster_centers_ = centers
        self.u_ = U
        self.labels_ = np.argmax(U, axis=1) # hard assignment = cluster with maximum membership
        return self
```

```
# Fit FCM with the same K as the other algorithms (m = 2 is the standard fuzzifier)
fcm = FuzzyCMeans(n_clusters=optimal_k, m=2.0, random_state=42)
fcm.fit(X_mall_scaled)
fcm_labels = fcm.labels_
fcm_centers = scaler.inverse_transform(fcm.cluster_centers_)

# ---- Evaluation of the hard partition induced by the fuzzy memberships ----
print(f"FCM Silhouette Score: {silhouette_score(X_mall_scaled, fcm_labels):.4f}")
print(f"FCM Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, fcm_labels):.4f}")
fpc = np.mean(np.sum(fcm.u_ ** 2, axis=1)) # fuzzy partition coefficient (1/K fuzzy ... 1 hard)
print(f"FCM Fuzzy Partition Coefficient (FPC): {fpc:.4f}")
```

Experiment 5: DBSCAN

Category: Density-based learning

1. Objective

Detect dense groups and noise with DBSCAN without specifying the number of clusters, on real spatial data.

2. Dataset Used

Real USGS earthquake catalogue 2023 (data/earthquakes.csv): 16,190 global events with magnitude ≥ 4.0 ; the experiment uses the 7,638 events with magnitude ≥ 4.5 . Coordinates are converted to radians and clustered with the haversine (great-circle) metric.

3. Required Libraries

scikit-learn, numpy, pandas, matplotlib

4. Brief Theory

DBSCAN uses a radius ϵ and minimum points MinPts. Points with $\geq$ MinPts neighbors within ϵ are core points; points within ϵ of a core are border points; the rest are noise (label -1). Clusters are chains of density-reachable core points, so arbitrary shapes can be discovered. A single global ϵ struggles when densities vary.

5. Procedure

1. Load the catalogue and filter to magnitude ≥ 4.5 (well-recorded events).
2. Convert latitude/longitude to radians for the haversine metric.
3. Choose ϵ from the k-distance graph ($k = 10$) — the knee at $0.03 \text{ rad} \approx 191 \text{ km}$.
4. Fit DBSCAN and count clusters/noise.
5. Show a parameter-sensitivity table for at least two settings.
6. Visualize epicenters and check the largest clusters against known seismic regions.

6. Reference Implementation

```
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
nbrs = NearestNeighbors(n_neighbors=10, metric='haversine').fit(X_rad)
k_distances = np.sort(nbrs.kneighbors(X_rad)[0][:, -1]) # knee ~0.03 rad
dbscan = DBSCAN(eps=0.03, min_samples=10, metric='haversine')
db_labels = dbscan.fit_predict(X_rad)
```

7. Results

Parameter sensitivity (min_samples = 10):

eps (rad)	approx km	clusters	noise %
0.02	127	70	21.2%
0.03 (selected)	191	55	13.5%
0.05	319	50	5.7%

Largest clusters (real-world validation): Philippines (1,629 events), Papua New Guinea (1,068), Tonga (947), Japan (895), South Sandwich Islands (253), Turkey (207). Silhouette on non-noise (3,000-event sample) = 0.326.

8. Discussion and Conclusion

DBSCAN recovers the global seismic structure without labels: dense chains along tectonic plate boundaries form clusters, while $\sim 1,000$ isolated intraplate events are noise. Smaller ϵ fragments clusters and labels more events as noise; larger ϵ merges boundaries and reduces noise. The single global ϵ is the main limitation, which motivates HDBSCAN.

9. Answers to Viva Questions

- **Why does DBSCAN not require K?** Clusters are grown from density-connected core points, so their number and shape emerge from the data instead of being fixed in advance.
- **What does label -1 mean?** The point is neither a core nor a border point — it lies in a sparse region and is treated as noise/outlier (here: isolated earthquakes).
- **Why can DBSCAN struggle with varying densities?** A single ϵ that is small enough for a dense cluster fragments a sparse one, while one large enough for the sparse cluster merges the dense ones — no global setting fits both.

10. Complete Code Listing

Full runnable program for this experiment, extracted from *notebooks/02_density_based_learning*.
 $\hookrightarrow$ *ipynb* (executed; results above are its actual output).

Listing 5.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms ----
from sklearn.cluster import DBSCAN, HDBSCAN
# ---- Nearest-neighbor search (used to pick epsilon) ----
from sklearn.neighbors import NearestNeighbors
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
 $\hookrightarrow$  default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Listing 5.2 — Load the real USGS earthquake catalogue (global, 2023)

```
# ---- Load the real USGS earthquake catalogue (global, 2023) ----
df = pd.read_csv('../data/earthquakes.csv')
print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
print(df.head(3).to_string(index=False))

# Keep well-recorded moderate+ events: magnitude >= 4.5
quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")

# ---- Convert epicenters to radians [latitude, longitude] ----
# scikit-learn's 'haversine' metric expects radian coordinates and computes
# great-circle distance on the sphere (correct for global data).
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
```

```
print("Coordinate range (deg): lat", quakes.latitude.min(), "to", quakes.latitude.max(),
      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())
```

Listing 5.3 — Choosing epsilon from the k-distance graph (haversine metric)

```
# ---- Choosing epsilon from the k-distance graph (haversine metric) ----
# Sorted distance to each event's k-th nearest neighbor. The knee marks the
# transition from dense seismic zones to sparse, isolated events.
min_samples = 10
nbrs = NearestNeighbors(n_neighbors=min_samples, metric='haversine').fit(X_rad)
distances, _ = nbrs.kneighbors(X_rad)
k_distances = np.sort(distances[:, -1]) # radians (0.03 rad = 191 km on Earth)

plt.figure(figsize=(9, 5))
plt.plot(k_distances, color='crimson', linewidth=2.5)
plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon Selection", fontsize=14, fontweight='bold'
        ↪ )
plt.xlabel("Earthquakes Sorted by Distance", fontsize=12)
plt.ylabel(f"{min_samples}-NN Distance (radians)", fontsize=12)
plt.axhline(y=0.03, color='navy', linestyle='--', label='Selected ε = 0.03 rad (~191 km)')
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 5.4 — Fit DBSCAN with the chosen epsilon and MinPts

```
# ---- Fit DBSCAN with the chosen epsilon and MinPts ----
# DBSCAN grows clusters from core events, connects them via density-reachability,
# and labels everything else as noise (-1).
eps_selected = 0.03 # radians -> 0.03 * 6371 km ~ 191 km
dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples, metric='haversine')
db_labels = dbscan.fit_predict(X_rad)

# Identify which events are core points (returned by the model) vs. border points
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise_db = np.sum(db_labels == -1)
print(f"DBSCAN: {n_clusters_db} clusters, {n_noise_db} noise events ({n_noise_db / len(X_rad) *
        ↪ 100:.1f}%)")

# ---- Visualize epicenters: clustered events in color, noise as black crosses ----
plt.figure(figsize=(12, 5.5))
palette = sns.color_palette("tab20", max(n_clusters_db, 1))
for k in range(n_clusters_db):
    mask = db_labels == k
    plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k % 20], alpha
        ↪ =0.75)

noise_mask = db_labels == -1
plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6, color='black', marker='x'
        ↪ , label='Noise (-1)')
plt.title(f"DBSCAN on Global Earthquakes 2023 ε={eps_selected} rad, {n_clusters_db} clusters",
        ↪ fontsize=13, fontweight='bold')
plt.xlabel("Longitude", fontsize=12)
plt.ylabel("Latitude", fontsize=12)
plt.legend(loc='lower left')
plt.tight_layout()
plt.show()
```

Listing 5.5 — Parameter sensitivity: DBSCAN under several (eps, min_samples) settings

```
# ---- Parameter sensitivity: DBSCAN under several (eps, min_samples) settings ----
```

```
# The manual requires at least two settings so the effect of eps can be discussed.
print("eps (rad)   approx km   clusters   noise %")
for eps_test in [0.02, 0.03, 0.05]:
    lbl = DBSCAN(eps=eps_test, min_samples=10, metric='haversine').fit_predict(X_rad)
    n_clusters_test = len(set(lbl)) - (1 if -1 in lbl else 0)
    noise_pct = (lbl == -1).mean() * 100
    print(f"   {eps_test:.2f}           {eps_test * 6371:5.0f}           {n_clusters_test:3d}           {noise_pct
↵ :5.1f}%")
```


Membership probabilities are high (> 0.9) for events deep inside seismic zones and lower near cluster edges. HDBSCAN's silhouette is roughly double DBSCAN's because adaptive density splits DBSCAN's long arc-shaped clusters (e.g., the whole Japan-Kuril chain) into compact, well-separated groups.

8. Discussion and Conclusion

HDBSCAN needs no ϵ and handles the varying density along plate boundaries better than DBSCAN; it is more conservative (21.5% vs 13.5% noise) because clusters must be stable across density scales. Increasing `min_cluster_size` reduces the number of small clusters; decreasing it fragments. The cost is a more complex algorithm and less transparent parameter meaning.

9. Answers to Viva Questions

- **What is hierarchical about HDBSCAN?** It constructs a full hierarchy of density-based clusters across all ϵ values, then condenses it by tracking cluster birth/death and keeping the most stable clusters.
- **What does membership probability mean?** A per-point confidence that it belongs to its assigned cluster, derived from the point's persistence in the cluster as density thresholds change (low values indicate borderline points).
- **Why is HDBSCAN useful for variable-density data?** Because clusters are selected by stability across density levels rather than by one global ϵ , so dense and sparse clusters can coexist.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/02_density_based_learning.ipynb` (executed; results above are its actual output).

Listing 6.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms ----
from sklearn.cluster import DBSCAN, HDBSCAN
# ---- Nearest-neighbor search (used to pick epsilon) ----
from sklearn.neighbors import NearestNeighbors
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
↳ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Listing 6.2 — Load the real USGS earthquake catalogue (global, 2023)

```
# ---- Load the real USGS earthquake catalogue (global, 2023) ----
df = pd.read_csv('../data/earthquakes.csv')
print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
print(df.head(3).to_string(index=False))
```

```
# Keep well-recorded moderate+ events: magnitude >= 4.5
quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")

# ---- Convert epicenters to radians [latitude, longitude] ----
# scikit-learn's 'haversine' metric expects radian coordinates and computes
# great-circle distance on the sphere (correct for global data).
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
print("Coordinate range (deg): lat", quakes.latitude.min(), "to", quakes.latitude.max(),
      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())
```

Listing 6.3 — Fit HDBSCAN (no global epsilon required)

```
# ---- Fit HDBSCAN (no global epsilon required) ----
# min_cluster_size = smallest group considered a cluster; min_samples controls conservativeness.
hdb = HDBSCAN(min_cluster_size=50, min_samples=10, metric='haversine', copy=False)
hdb_labels = hdb.fit_predict(X_rad)

n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise_hdb = np.sum(hdb_labels == -1)
print(f"HDBSCAN: {n_clusters_hdb} clusters, {n_noise_hdb:,} noise events ({n_noise_hdb / len(X_rad) *
  ↳ 100:.1f}%)")

# ---- Visualize the HDBSCAN partition ----
plt.figure(figsize=(12, 5.5))
palette = sns.color_palette("tab20", max(n_clusters_hdb, 1))
for k in range(n_clusters_hdb):
    mask = hdb_labels == k
    plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k % 20], alpha
  ↳ =0.75)

noise_mask = hdb_labels == -1
plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6, color='black', marker='x'
  ↳ , label='Noise (-1)')
plt.title(f"HDBSCAN on Global Earthquakes 2023 ({n_clusters_hdb} clusters, no  $\epsilon$ )", fontsize=13,
  ↳ fontweight='bold')
plt.xlabel("Longitude", fontsize=12)
plt.ylabel("Latitude", fontsize=12)
plt.legend(loc='lower left')
plt.tight_layout()
plt.show()
```

Experiment 7: Self-training for Semi-supervised Learning

Category: Semi-supervised learning

1. Objective

Train a classifier with a small labeled subset and a larger unlabeled subset by iteratively adding high-confidence pseudo-labels; compare with a supervised baseline and the fully supervised ceiling.

2. Dataset Used

Breast Cancer Wisconsin (569×30 , binary). Stratified 70/30 split: 398 training / 171 test. Only 15% of training labels (61 samples) are kept; 337 samples are masked as unlabeled (-1).

3. Required Libraries

scikit-learn, numpy

4. Brief Theory

Self-training fits a base classifier on labeled data, predicts class probabilities for the unlabeled pool, converts sufficiently confident predictions (max probability $\geq$ threshold) to pseudo-labels, adds them to the labeled set and repeats. Errors can propagate (confirmation bias), so the confidence threshold and convergence monitoring matter.

5. Procedure

1. Split the data stratified; scale features (fit on training data only).
2. Mask 85% of training labels to -1.
3. Train a supervised baseline SVC on the labeled 61 samples.
4. Wrap an RBF SVC in `SelfTrainingClassifier(threshold=0.80)`, fit on labeled + unlabeled, monitor iterations.
5. Train a fully supervised SVC on all 398 labels as the ceiling.
6. Compare accuracy/F1 on the untouched test set.

6. Reference Implementation

```
self_training_svc = SelfTrainingClassifier(  
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),  
    threshold=0.80, criterion='threshold', max_iter=15, verbose=True)  
self_training_svc.fit(X_train_full_scaled, y_train_semi)  # labels + unlabeled (-1)
```

7. Results

Setting	Accuracy	F1	Training samples
Supervised baseline (15% labeled)	0.9298	0.9469	61
Self-training (semi-supervised)	0.9415	0.9554	382
Fully supervised ceiling (100%)	0.9766	0.9813	398

Pseudo-labels added per iteration: +292, +22, +5, +2, then termination no_change (321 new labels total). The decaying additions indicate stable convergence rather than runaway error accumulation.

8. Discussion and Conclusion

Using no extra human labels, self-training improves accuracy by +1.2 points over the baseline, closing part of the gap to the fully supervised model. The remaining gap is the cost of noisy pseudo-labels. Convergence was quick (4 useful iterations) because the threshold 0.80 admitted mostly correct predictions.

9. Answers to Viva Questions

- **What is a pseudo-label?** A predicted label for an unlabeled sample that the model accepts as training target because its confidence exceeds the threshold.
- **What is confirmation bias in self-training?** The model reinforces its own mistakes: wrong confident predictions enter the training set, make the model more confident about similar errors, and can degrade performance. High thresholds and validation monitoring mitigate this.
- **Why must the test labels remain hidden during training?** Otherwise the evaluation is contaminated — test data would influence the model, and the reported accuracy would no longer estimate generalization.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/03_semi_supervised_learning`.
 ↪ `ipynb` (executed; results above are its actual output).

Listing 7.1 — Suppress a known sklearn deprecation notice for SVC(probability=True)

```
# ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    ↪ confusion_matrix, classification_report

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-learn still supports SVC(probability=True) but warns; silence that specific
    ↪ notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated", category=
    ↪ FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")

# ---- Load the Breast Cancer Wisconsin dataset ----
```

```

data = load_breast_cancer()
X_full = data.data           # 30 continuous diagnostic features
y_full = data.target         # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-learn convention for "unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)

print(f"Total Training Samples: {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples: {len(y_test)}")

```

Listing 7.2 — Experiment 1: Self-Training with an RBF-kernel SVC

```

# ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 labeled samples
mask_labeled = (y_train_semi != -1) # boolean mask of the labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80, # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold', # use the confidence-threshold selection rule
    max_iter=15,
    verbose=True # print how many labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

# 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)
acc_full_svc = accuracy_score(y_test, y_pred_full_svc)

```

```
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc}
↪ :.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")
```

Listing 7.3 — Inspect how self-training converged

```
# ---- Inspect how self-training converged ----
# n_iter_ : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_ : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(
↪ y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")
```

Listing 7.4 — Compact comparison table (SVC)

```
# ---- Compact comparison table (SVC) ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score": f1_baseline_svc
↪ ,
    "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score":
↪ f1_semi_svc,
    "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score":
↪ f1_full_svc,
    "Training Samples": len(y_train_full)},
]
summary_df = pd.DataFrame(summary_data).set_index("Model")
display(summary_df.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))
```

Experiment 8: Random Forest Regression (RFR)

Category: Ensemble learning

1. Objective

Predict a continuous target using an ensemble of randomized decision trees; evaluate with MAE, RMSE and R^2 and inspect out-of-bag validation.

2. Dataset Used

California Housing (`fetch_california_housing`), 5,000-row random sample, 8 features, target = median house value. 80/20 train/test split (4,000/1,000).

3. Required Libraries

scikit-learn, numpy, pandas

4. Brief Theory

Random Forest Regression averages predictions of B trees trained on bootstrap samples, each split considering a random feature subset. Averaging decorrelated trees reduces variance. Out-of-bag samples (not drawn in a bootstrap) give a free internal validation estimate.

5. Procedure

1. Load and sample the dataset; split train/test.
2. Train `RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True)`.
3. Predict the test set; compute R^2 , RMSE, MAE.
4. Report the OOB R^2 and compare with the test R^2 to check overfitting.

6. Reference Implementation

```
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12,
                              oob_score=True, random_state=42, n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)
print(rf_reg.oob_score_, r2_score(y_test_reg, rf_reg.predict(X_test_reg)))
```

7. Results

Metric	Value
OOB R^2	0.7513
Test R^2	0.7415
Test RMSE	0.5908 (\$100k units $\approx$ \$59k)
Test MAE	0.3999 (\$100k units $\approx$ \$40k)

OOB R^2 (0.7513) is very close to the test R^2 (0.7415), so the model is not overfitting. Feature importances are dominated by median income and location (latitude/longitude), consistent with housing economics.

8. Discussion and Conclusion

RFR provides a strong, low-maintenance baseline for tabular regression: no feature scaling is required, it captures non-linearities and interactions, and OOB gives free validation. Gradient boosting (Experiments 10-12) reaches higher R^2 on the same data at the cost of more tuning.

9. Answers to Viva Questions

- **Why does a forest reduce variance?** Averaging B approximately uncorrelated trees gives $\text{Var} \approx \rho\sigma^2 + (1-\rho)\sigma^2/B$; as B grows the second term vanishes, so predictions are more stable than a single tree's.
- **What is bagging?** Bootstrap aggregating: train each model on a bootstrap sample of the data and combine predictions (average for regression, vote for classification).
- **Why can feature importance be misleading?** Impurity-based importance is biased toward high-cardinality/continuous features, is computed on training data (no causality), and correlated features share/dilute importance arbitrarily.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/04_ensemble_learning.ipynb` (executed; results above are its actual output).

Listing 8.1 — Core libraries

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
    ↪ AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
    ↪ f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 8.2 — Load the California Housing dataset (real-world regression benchmark)

```
# ---- Load the California Housing dataset (real-world regression benchmark) ----
housing = fetch_california_housing(as_frame=True)
# Sample 5,000 rows so every ensemble trains quickly while remaining representative
df_reg = housing.frame.sample(n=5000, random_state=42)

X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
y_reg = df_reg['MedHouseVal']         # target: median house value

# ---- Train/test split (80% / 20%) ----
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.20, random_state=42
)

print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples: {X_test_reg.shape[0]}")
df_reg.head()
```

Listing 8.3 — Random Forest Regression (bagging of decision trees)

```
# ---- Random Forest Regression (bagging of decision trees) ----
```

```
# n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag samples
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True, random_state=42,
    ↪ n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_rf = rf_reg.predict(X_test_reg)
r2_rf = r2_score(y_test_reg, y_pred_rf)           # explained variance (1.0 = perfect)
rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf) # error in target units
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)    # robust average error

print(f"Random Forest Regressor Results:")
print(f" - Out-of-Bag (OOB) R2 Score: {rf_reg.oob_score_:.4f}")
print(f" - Test R2 Score: {r2_rf:.4f}")
print(f" - Test RMSE: {rmse_rf:.4f}")
print(f" - Test MAE: {mae_rf:.4f}")
```

Experiment 9: Random Forest Classification (RFC)

Category: Ensemble learning

1. Objective

Classify observations with an ensemble of randomized decision trees using a stratified split and report accuracy, precision, recall, F1 and the confusion matrix.

2. Dataset Used

Heart Disease (data/heart_disease.csv, 303 rows, 13 features plus target). Categorical variables are one-hot encoded for the forest; stratified 75/25 split (227/76).

3. Required Libraries

scikit-learn, numpy, pandas

4. Brief Theory

Random Forest Classification aggregates tree votes: $\hat{C}(x) = \text{argmax}_k \sum_b I(T_b(x) = k)$. Bootstrap sampling plus feature subsampling decorrelates trees, reducing variance relative to a single decision tree. The class-probability output (fraction of votes) enables ROC-AUC.

5. Procedure

1. Load the heart data; one-hot encode categorical features.
2. Use a stratified 75/25 split to preserve the disease ratio.
3. Train `RandomForestClassifier(n_estimators=120, max_depth=8)`.
4. Predict labels and probabilities on the test set.
5. Compute accuracy, F1 and ROC-AUC.

6. Reference Implementation

```
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[: , 1]
```

7. Results

```
____
() ()
* *
0.5060000000000001
____
() ()
* *
0.5060000000000001
0.7763
cu-
racy
```

```

—
() ()
* *
0.5060000000000001
—
() ()
* *
0.5060000000000001
ci-(F1)
sion
/
Re-
call
/
F1
() ()
* *
0.5060000000000001
AUC
() ()
* *
0.5060000000000001
fu-P/FN/TP
store-
maported
trix
(teste-
76book
sam-
ples)
—

```

ROC-AUC 0.87 indicates good ranking quality on a small dataset; accuracy 0.776 is typical for heart-disease prediction with these features and reflects the problem's noise, not a modelling failure.

8. Discussion and Conclusion

RFC gives a robust classifier with minimal preprocessing and interpretable feature importances. On 227 training rows the variance-reduction of bagging is important; class imbalance (about 45% positive here) is mild, but F1 and AUC are safer than accuracy alone.

9. Answers to Viva Questions

- **RFR vs RFC?** Regression averages the trees' numeric outputs; classification takes a majority vote (and averages class probabilities for scoring).
- **What is majority voting?** Each tree votes for one class and the most frequent class becomes the ensemble prediction (soft voting averages probabilities instead).
- **How can class imbalance affect accuracy?** With a rare positive class, a model can predict the majority class everywhere and still show high accuracy; precision/recall/F1 and AUC expose the failure.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/04_ensemble_learning.ipynb` (executed; results above are its actual output).

Listing 9.1 — Core libraries

```

# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
    ↪ AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
    ↪ f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

```

Listing 9.2 — Load the Heart Disease dataset (real-world classification benchmark)

```

# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}
    ↪ ")

```

Experiment 10: XGBoost

Category: Ensemble learning / Gradient boosting

1. Objective

Train a gradient-boosted tree classifier and analyze the effect of boosting hyperparameters.

2. Dataset Used

Heart Disease (data/heart_disease.csv), same stratified 75/25 split as RFC for a fair comparison.

3. Required Libraries

xgboost, scikit-learn, numpy, pandas

4. Brief Theory

XGBoost builds trees sequentially, each fitting the 1st/2nd-order gradients (g_i , h_i) of the loss: $L \approx \sum [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$, with $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$. Controls: `learning_rate`, `n_estimators`, `max_depth`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`.

5. Procedure

1. Prepare the one-hot encoded heart data and the same stratified split.
2. Fit `XGBClassifier(n_estimators=120, learning_rate=0.08, max_depth=4, subsample=0.8, colsample_bytree=0.8, eval_metric='logloss')`.
3. Predict labels and probabilities.
4. Report accuracy, F1 and ROC-AUC.
5. Discuss the complexity/generalization trade-off.

6. Reference Implementation

```
xgb_clf = xgb.XGBClassifier(n_estimators=120, learning_rate=0.08, max_depth=4,
                           subsample=0.8, colsample_bytree=0.8,
                           random_state=42, eval_metric='logloss')
xgb_clf.fit(X_train_clf, y_train_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]
```

7. Results

Metric	Value
Accuracy	0.7763
F1	0.8132
ROC-AUC	0.8509

XGBoost matches RFC on accuracy with a slightly higher F1 but a slightly lower AUC on this small test set (76 samples). Differences of ~0.01-0.02 are within noise for this dataset size.

8. Discussion and Conclusion

Gradient boosting is the strongest family for tabular data in general (see regression results: XGBoost R^2 0.8025 vs RFR 0.7415), but on this small heart dataset with fixed hyperparameters it does not clearly beat simpler ensembles. Lower learning rates with more trees usually improve generalization at higher training cost.

9. Answers to Viva Questions

- **Bagging vs boosting?** Bagging trains models in parallel on bootstrap samples and combines them to reduce variance; boosting trains models sequentially so each corrects the previous ensemble's errors, mainly reducing bias.
- **What does learning rate do?** It shrinks each tree's contribution. Smaller values need more trees but usually generalize better; larger values fit faster and can overfit.
- **Why can boosting overfit?** Later trees keep fitting remaining errors, including label noise, and the ensemble can become too complex; regularization (depth, λ , subsample, early stopping) controls this.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/04_ensemble_learning.ipynb` (executed; results above are its actual output).

Listing 10.1 — Core libraries

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
    ↪ AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
    ↪ f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 10.2 — Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")
```

```
# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}
↪ ")
```

Listing 10.3 — XGBoost classification on the same stratified split

```
# ---- XGBoost classification on the same stratified split ----
xgb_clf = xgb.XGBClassifier(
    n_estimators=120, learning_rate=0.08, max_depth=4,
    subsample=0.8, colsample_bytree=0.8,
    random_state=42, eval_metric='logloss'
)
xgb_clf.fit(X_train_clf, y_train_clf)

y_pred_xgb_c = xgb_clf.predict(X_test_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]

acc_xgb_c = accuracy_score(y_test_clf, y_pred_xgb_c)
f1_xgb_c = f1_score(y_test_clf, y_pred_xgb_c)
auc_xgb_c = roc_auc_score(y_test_clf, y_prob_xgb_c)
print(f"XGBoost Classifier: Accuracy = {acc_xgb_c:.4f}, F1 = {f1_xgb_c:.4f}, ROC-AUC = {auc_xgb_c:.4f}
↪ ")
```

Experiment 11: AdaBoost

Category: Ensemble learning

1. Objective

Build an adaptive boosting classifier and study how weak learners combine into a stronger ensemble.

2. Dataset Used

Heart Disease (data/heart_disease.csv), same stratified 75/25 split.

3. Required Libraries

scikit-learn, numpy, pandas

4. Brief Theory

AdaBoost trains weak learners sequentially on re-weighted data: misclassified samples receive larger weights. At round m the learner weight is $\alpha_m = \frac{1}{2} \ln((1 - \epsilon_m)/\epsilon_m)$, and sample weights update as $w_i \leftarrow w_i \exp(-\alpha_m y_i h_m(x_i))$. The final prediction is a weighted vote. Exponential loss makes it sensitive to noisy points.

5. Procedure

1. Prepare the heart data and stratified split.
2. Fit `AdaBoostClassifier(n_estimators=100, learning_rate=0.1)` (stumps by default).
3. Predict labels and probabilities on the test set.
4. Report accuracy, F1 and ROC-AUC; compare with RFC and XGBoost.

6. Reference Implementation

```
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[: , 1]
```

7. Results

Metric	Value
Accuracy	0.7895 (best of the four classifiers)
F1	0.8182 (best)
ROC-AUC	0.8714 (best)

AdaBoost with shallow stumps slightly outperforms the deeper models on this small dataset, which is a common outcome when the signal is mostly additive and the sample size is limited.

8. Discussion and Conclusion

AdaBoost remains a strong, cheap baseline: stump ensembles are fast, need little tuning, and here achieve top accuracy/F1/AUC. Its weakness is sensitivity to label noise and outliers (weights grow exponentially for hard points); gradient boosting with regularization is more robust on noisier data.

9. Answers to Viva Questions

- **Why use weak learners?** Individually they are simple and low-variance; boosting combines many of them so the ensemble can fit complex boundaries while keeping each step's variance low.
- **How does AdaBoost focus on difficult examples?** After each round, weights of misclassified samples increase, so the next learner optimizes a distribution that emphasizes current errors.
- **What is the effect of noisy labels?** Mislabeled points look “hard”, receive ever-larger weights, and can dominate training, degrading accuracy — AdaBoost is known to be noise-sensitive.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/04_ensemble_learning.ipynb` (executed; results above are its actual output).

Listing 11.1 — Core libraries

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
    ↪ AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
    ↪ f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 11.2 — Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")
```

```
# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}
↪ ")
```

Listing 11.3 — AdaBoost classification

```
# ---- AdaBoost classification ----
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)

y_pred_ada_c = ada_clf.predict(X_test_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[:, 1]

acc_ada_c = accuracy_score(y_test_clf, y_pred_ada_c)
f1_ada_c = f1_score(y_test_clf, y_pred_ada_c)
auc_ada_c = roc_auc_score(y_test_clf, y_prob_ada_c)
print(f"AdaBoost Classifier: Accuracy = {acc_ada_c:.4f}, F1 = {f1_ada_c:.4f}, ROC-AUC = {auc_ada_c:.4f}
↪ ")
```

Experiment 12: CatBoost

Category: Ensemble learning

1. Objective

Train a gradient-boosted model with native handling of categorical features and inspect correct categorical declaration.

2. Dataset Used

Heart Disease (data/heart_disease.csv): continuous features (age, trestbps, chol, thalach, oldpeak) plus categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal). The raw (unencoded) table is used for CatBoost.

3. Required Libraries

catboost, pandas, scikit-learn

4. Brief Theory

CatBoost uses ordered boosting (leaf values computed on random permutations) to avoid target leakage/prediction shift, and builds oblivious (symmetric) trees. Categorical features are encoded internally with ordered target statistics, so `cat_features` must be declared correctly and the data split must not leak the target.

5. Procedure

1. Identify categorical columns in the raw heart table and cast them to string.
2. Use the same stratified 75/25 split on the raw features.
3. Fit `CatBoostClassifier(iterations=150, depth=5, cat_features=categorical_cols)`.
4. Predict labels/probabilities; report accuracy, F1, ROC-AUC.
5. Explain why target-statistic encoding can beat one-hot encoding.

6. Reference Implementation

```
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
cb_clf = cb.CatBoostClassifier(iterations=150, learning_rate=0.08, depth=5,
                              cat_features=categorical_cols, random_seed=42, verbose=False)
cb_clf.fit(X_tr_cb, y_tr_cb)
```

7. Results

Metric	Value
Accuracy	0.7632
F1	0.7955
ROC-AUC	0.8551

CatBoost performs at a similar level to the other ensembles without any manual one-hot encoding — its native categorical handling did the encoding internally. On this tiny dataset, the advanced target statistics cannot show their full advantage (they matter most with high-cardinality categories, e.g. thousands of city/user IDs).

8. Discussion and Conclusion

CatBoost is the ensemble of choice when tables mix numeric and categorical columns: it removes manual encoding, reduces target leakage, and remains strong out-of-the-box. Here its metrics sit within noise of RFC/XGBoost/AdaBoost because the categorical cardinalities are small (2-4 values).

9. Answers to Viva Questions

- **How does CatBoost treat categorical features?** It computes ordered target statistics (mean target per category on random permutations with prior smoothing) rather than one-hot vectors.
- **Why should categories not be encoded using target leakage?** Plain target encoding uses each row's own target when encoding that row, which leaks label information and inflates training performance; ordered statistics avoid this.
- **CatBoost vs ordinary one-hot encoding?** One-hot explodes dimensionality for high-cardinality features and loses category statistics; CatBoost encodes compactly and exploits target-category relationships with leakage control.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/04_ensemble_learning.ipynb` (executed; results above are its actual output).

Listing 12.1 — Core libraries

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
    ↪ AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
    ↪ f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 12.2 — Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
```

```

X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}
↪ ")

```

Listing 12.3 — CatBoost classification with native categorical features

```

# ---- CatBoost classification with native categorical features ----
# Identify categorical columns in the raw (unencoded) heart data
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
X_raw_cb = X_clf_raw.copy()
for col in categorical_cols:
    X_raw_cb[col] = X_raw_cb[col].astype(str)

# Same stratified split on the raw feature table
X_tr_cb, X_te_cb, y_tr_cb, y_te_cb = train_test_split(
    X_raw_cb, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

cb_clf = cb.CatBoostClassifier(
    iterations=150, learning_rate=0.08, depth=5,
    cat_features=categorical_cols, random_seed=42, verbose=False
)
cb_clf.fit(X_tr_cb, y_tr_cb)

y_pred_cb_c = cb_clf.predict(X_te_cb)
y_prob_cb_c = cb_clf.predict_proba(X_te_cb)[:, 1]

acc_cb_c = accuracy_score(y_te_cb, y_pred_cb_c)
f1_cb_c = f1_score(y_te_cb, y_pred_cb_c)
auc_cb_c = roc_auc_score(y_te_cb, y_prob_cb_c)
print(f"CatBoost Classifier: Accuracy = {acc_cb_c:.4f}, F1 = {f1_cb_c:.4f}, ROC-AUC = {auc_cb_c:.4f}
↪ ")

```

Experiment 13: Multilayer Perceptron (MLP)

Category: Neural networks

1. Objective

Implement feed-forward neural networks for handwritten-digit classification and study architecture and regularization.

2. Dataset Used

Digits (scikit-learn): 1,797 samples, 8×8 grayscale images (64 features), 10 classes. Stratified 70/15/15 train/validation/test split; standardized (scaler fit on train only).

3. Required Libraries

scikit-learn, numpy, pandas, matplotlib, PyTorch (torch)

4. Brief Theory

An MLP stacks affine transformations with non-linear activations: $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$, trained by backpropagation to minimize cross-entropy. BatchNorm stabilizes activations, Dropout regularizes, and an LR scheduler adapts the step size.

5. Procedure

1. Load digits; split 70/15/15; standardize.
2. Train `MLPClassifier(hidden_layer_sizes=(128,64), ReLU, Adam, alpha=0.001, early_stopping=True)`.
3. Build a custom PyTorch MLP: 64→128→64→10 with BatchNorm + Dropout(0.25/0.20), CrossEntropyLoss, Adam (lr 0.003, weight decay 1e-4), 50 epochs, batch 32, ReduceLROnPlateau.
4. Train with the validation loop; plot the loss curve.
5. Evaluate the test set; report the classification report.

6. Reference Implementation

```
“python mlp_sklearn = MLPClassifier(hidden_layer_sizes=(128, 64), activation='relu', solver='adam',
alpha=0.001, batch_size=64, learning_rate_init=0.005, max_iter=150, early_stopping=True,
validation_fraction=0.15) ## 10. Complete Code Listing
```

*Full runnable program for this experiment, extracted from notebooks/05_multilayer_perceptron.
↪ ipynb (executed; results above are its actual output).*

Listing 13.1 — Core numerical and plotting libraries

```
“python
```

Experiment 14: Recurrent Neural Network (RNN)

Category: Sequence modeling

1. Objective

Build recurrent networks for sequential data: (a) sequence classification and (b) time-series forecasting, comparing vanilla RNN with LSTM.

2. Datasets Used

- (a) Synthetic sequence benchmark: 1,200 sequences of length 20, label = 1 if the sum is positive (1,000 train / 200 test). (b) Monthly Airline Passengers (data/airline_passengers.csv $\hookrightarrow$): 144 months, 12-month sliding windows, chronological 80/20 split.

3. Required Libraries

PyTorch, numpy, pandas, matplotlib (the manual's TensorFlow/Keras example is implemented equivalently in PyTorch, the framework installed in this environment)

4. Brief Theory

An RNN maintains a hidden state $h_t = \tanh(W x_t + U h_{t-1} + b)$ so the output depends on the sequence history; training uses backpropagation through time. Vanilla RNNs suffer from vanishing gradients over long sequences; LSTM gates (forget/input/output + additive cell state) preserve long-range information.

5. Procedure

1. Generate the synthetic sequence classification data and split 1,000/200.
2. Build RNN and LSTM classifiers (hidden 32, linear sigmoid output); train 10 epochs with Adam and BCEWithLogitsLoss.
3. Report test accuracy for both.
4. For forecasting: normalize the passenger series, build 12-month windows, split chronologically.
5. Train RNN and LSTM forecasters (hidden 64, 2 layers, MSE, Adam, 120 epochs).
6. Report RMSE/MAE and plot the forecasts.

6. Reference Implementation

“python ## 10. Complete Code Listing

*Full runnable program for this experiment, extracted from notebooks/06_recurrent_neural_network.
 $\hookrightarrow$ ipynb (executed; results above are its actual output).*

Listing 14.1 — Core numerical and plotting libraries

“python

Experiment 15: Self-Organizing Map (SOM)

Category: Unsupervised neural learning

1. Objective

Map high-dimensional observations onto a low-dimensional grid while preserving neighborhood structure; evaluate with quantization and topographic error.

2. Dataset Used

UCI Wine: 178 samples, 13 chemical features, 3 cultivars (labels used only for visualization). MinMax-scaled to [0,1].

3. Required Libraries

minisom, numpy, pandas, matplotlib

4. Brief Theory

A SOM has a grid of neurons with weight vectors. For each input the Best Matching Unit (BMU) is found (minimum distance), then the BMU and its Gaussian neighborhood are moved toward the input: $w_j \leftarrow w_j + \alpha(t) h_{cj}(t) (x - w_j)$, with decaying $\alpha(t)$ and $\sigma(t)$. The result is a topology-preserving 2D projection; the U-Matrix visualizes cluster boundaries.

5. Procedure

1. Load wine; MinMax scale.
2. Create a 12×12 SOM with Gaussian neighborhood ($\sigma=1.5$, $lr=0.5$); initialize weights with PCA.
3. Train for 5,000 random samples.
4. Compute quantization error (average sample→BMU distance) and topographic error (fraction of non-adjacent top-2 BMUs).
5. Plot the U-Matrix with each sample projected onto its BMU (marker = true cultivar).
6. Discuss grid size and σ .

6. Reference Implementation

```
som = MiniSom(x=12, y=12, input_len=13, sigma=1.5, learning_rate=0.5,
              neighborhood_function='gaussian', random_seed=42)
som.pca_weights_init(X)
som.train_random(data=X, num_iteration=5000)
qe = som.quantization_error(X); te = som.topographic_error(X)
```

7. Results

Metric	Value
Initial quantization error	0.4447
Final quantization error	0.1965
Topographic error	0.0393 (~4% foldings)

On the U-Matrix, the three wine cultivars occupy contiguous regions separated by bright ridges, even though the SOM never saw the labels — evidence of unsupervised structure discovery.

8. Discussion and Conclusion

The map halved its quantization error during training and preserves topology well (topographic error < 0.05). PCA initialization gave a well-unfolded map from the start. A larger grid reduces quantization error but risks fragmenting clusters; σ/lr must decay for stable convergence.

9. Answers to Viva Questions

- **SOM vs K-means?** Both compress data by prototypes, but K-means only finds centroids while SOM arranges them on a grid so that neighboring neurons represent similar inputs (topology preservation).
- **What is a BMU?** The neuron whose weight vector is closest to the input, i.e. the winner of the competition.
- **Why is SOM called a topology-preserving map?** Because the update neighborhood ensures nearby grid positions end up with similar weight vectors, so high-dimensional neighborhoods are approximately preserved in 2D.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/07_self_organizing_map.ipynb` (executed; results above are its actual output).

Listing 15.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + scaling ----
from sklearn.datasets import load_wine
from sklearn.preprocessing import MinMaxScaler
# ---- MiniSom: lightweight Kohonen Self-Organizing Map implementation ----
from minisom import MiniSom

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
↳ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("MiniSom and libraries loaded successfully!")

# ---- Load the Wine recognition dataset ----
wine = load_wine()
X_raw = wine.data          # 178 samples x 13 chemical features
y = wine.target            # 3 wine cultivars (only used for visualization - SOM is unsupervised)
feature_names = wine.feature_names
target_names = wine.target_names

print(f"Samples: {X_raw.shape[0]}, Features: {X_raw.shape[1]}, Classes: {target_names.tolist()}")

# ---- MinMax scaling to [0, 1] ----
# SOM weight vectors live in the same range as the inputs, so scaling keeps learning stable.
scaler = MinMaxScaler()
X = scaler.fit_transform(X_raw)
```

Listing 15.2 — Configure the Kohonen grid

```
# ---- Configure the Kohonen grid ----
grid_x, grid_y = 12, 12 # 12x12 grid = 144 neurons (fewer than the 178 samples)
input_len = X.shape[1] # 13 chemical features per neuron weight vector

som = MiniSom(
    x=grid_x,
    y=grid_y,
    input_len=input_len,
    sigma=1.5, # initial neighborhood radius
    learning_rate=0.5, # initial learning rate
    neighborhood_function='gaussian', # smooth Gaussian neighborhood kernel
    random_seed=42
)

# ---- Initialize weights with PCA for topologically faithful, faster convergence ----
som.pca_weights_init(X)
print("Initial Quantization Error:", som.quantization_error(X))

# ---- Train: 5000 random samples presented to the map ----
som.train_random(data=X, num_iteration=5000, verbose=False)
final_qe = som.quantization_error(X) # average distance from samples to their BMU
final_te = som.topographic_error(X) # fraction of samples whose 2nd BMU is not a grid neighbor

print(f"Training complete after 5,000 iterations!")
print(f" - Final Quantization Error (mean distance to BMU): {final_qe:.4f}")
print(f" - Final Topographic Error (proportion of foldings): {final_te:.4f}")
```

Listing 15.3 — Plot the U-Matrix (unified distance map)

```
# ---- Plot the U-Matrix (unified distance map) ----
# Each cell = average distance between a neuron and its immediate grid neighbors.
# Dark valleys = dense clusters; bright ridges = boundaries between clusters.
plt.figure(figsize=(10, 8))
plt.pcolor(som.distance_map().T, cmap='bone_r', alpha=0.9)
cbar = plt.colorbar()
cbar.set_label("Normalized Inter-Neuron Distance (U-Matrix)", fontsize=11)

# ---- Project every sample onto its Best Matching Unit (BMU) ----
markers = ['o', 's', '^']
colors = ['red', 'green', 'blue']

for idx, x_vec in enumerate(X):
    w = som.winner(x_vec) # grid coordinates of the best matching neuron for this sample
    plt.plot(
        w[0] + 0.5, # +0.5 centers the marker inside the pcolor cell
        w[1] + 0.5,
        markers[y[idx]], # marker shape encodes the true cultivar
        markerfacecolor='None',
        markeredgecolor=colors[y[idx]],
        markersize=10,
        markeredgewidth=2
    )

# Build a legend for the three cultivars
for c_idx, c_name in enumerate(target_names):
    plt.plot([], [], marker=markers[c_idx], color=colors[c_idx], linestyle='None',
             label=f'Cultivar {c_name}', markersize=9, markeredgewidth=2)

plt.title("SOM U-Matrix with Projected Wine Cultivars", fontsize=14, fontweight='bold')
plt.legend(bbox_to_anchor=(1.25, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



```

—
() ()
* *
0.5060000000000001
—
() ()
* *
0.5060000000000001
log(1,859
likelihood
() ()
* *
0.5060000000000001
Bull
(low
vol),
Side-
ways
(medium),
Bear
(high
vol)
af-
ter
sort-
ing
—

```

The colored DAX chart shows the model tracking real episodes: the 1990s bull phases, corrections such as 1994, and the high-volatility cluster around the 1998 crisis. The transition-matrix diagonal is dominant, confirming regime persistence.

8. Discussion and Conclusion

The 3-state Gaussian HMM extracts interpretable, persistent market regimes from real data without labels. Its main assumptions are the Markov property (next state depends only on the current one) and Gaussian emissions; return fat tails and regime duration dependence motivate extensions (t-emissions, semi-Markov models).

9. Answers to Viva Questions

- **What is hidden in an HMM?** The state sequence: we observe emissions (returns) but not the underlying regime that generated them.
- **Viterbi vs forward algorithm?** Viterbi returns the single most likely state path (max over paths); the forward algorithm computes the total probability $P(X | \lambda)$ by summing over all paths.
- **What is the Markov assumption?** The future state depends on the past only through the present state: $P(z_t | z_{1:t-1}) = P(z_t | z_{t-1})$.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/08_hidden_markov_model.ipynb` (executed; results above are its actual output).

Listing 16.1 — Core libraries

```

# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from hmmlearn.hmm import GaussianHMM

# Styling setup

```

```
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
↳ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

# ---- Load the real DAX stock index (daily closes 1991-1998) ----
df_market = pd.read_csv('../data/stock_index.csv')
dates = pd.to_datetime(df_market['Date'])
print(f"Trading days: {len(df_market)} | {dates.iloc[0].date()} to {dates.iloc[-1].date()}")
print(df_market.head(3).to_string(index=False))

returns = df_market['Daily_Return'].values.reshape(-1, 1) # HMM observations
prices = df_market['Close'].values # only used for visualization
```

Listing 16.2 — Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm

```
# ---- Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm ----
n_components = 3
hmm_model = GaussianHMM(
    n_components=n_components, # Bull / Sideways / Bear regimes
    covariance_type="full", # each state has its own full covariance Gaussian emission
    n_iter=200, # EM iterations
    random_state=42,
    verbose=False
)

hmm_model.fit(returns)
print("EM Training Converged:", hmm_model.monitor_.converged)
print(f"Model Log-Likelihood: {hmm_model.score(returns):.2f}")

# ---- Decode the most likely hidden state sequence with the Viterbi algorithm ----
hidden_states = hmm_model.predict(returns)

# ---- Sort regimes by volatility so labels have a consistent economic meaning ----
# Regime 0: Low Volatility (Bull) | Regime 1: Moderate (Sideways) | Regime 2: High (Bear/Crisis)
volatilities = [np.sqrt(hmm_model.covars_[i][0][0]) for i in range(n_components)]
state_order = np.argsort(volatilities) # ascending
↳ volatility
state_mapping = {old_state: new_state for new_state, old_state in enumerate(state_order)}

sorted_hidden_states = np.array([state_mapping[s] for s in hidden_states]) # relabel sequence
sorted_means = [hmm_model.means_[old][0] for old in state_order] # reordered means
sorted_stds = [np.sqrt(hmm_model.covars_[old][0][0]) for old in state_order] # reordered std devs

# ---- Reorder the transition matrix to match the sorted regimes ----
sorted_transmat = np.zeros((n_components, n_components))
for i in range(n_components):
    for j in range(n_components):
        sorted_transmat[i, j] = hmm_model.transmat_[state_order[i], state_order[j]]

regime_names = ["Bull (Low Vol)", "Sideways (Med Vol)", "Bear (High Vol)"]
```

Listing 16.3 — Transition probability matrix

```
# ---- Transition probability matrix ----
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.heatmap(sorted_transmat, annot=True, fmt='.3f', cmap='Blues', ax=ax,
            xticklabels=regime_names, yticklabels=regime_names, cbar=False)
ax.set_title("HMM Transition Probability Matrix ($A_{ij}$)", fontweight='bold')
ax.set_xlabel("Transition To Regime")
ax.set_ylabel("Current Regime")
plt.tight_layout()
plt.show()
```

```

# ---- Viterbi-decoded regimes over the price series ----
colors = ['forestgreen', 'royalblue', 'crimson']
x = np.arange(len(prices))

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 7), sharex=True, gridspec_kw={'height_ratios': [3,
↪ 1]})

# Top: color every consecutive price segment by its decoded regime
for i in range(len(prices) - 1):
    ax1.plot([x[i], x[i + 1]], [prices[i], prices[i + 1]], color=colors[sorted_hidden_states[i]], lw
↪ =1.2)
for i, name in enumerate(regime_names):
    ax1.plot([], [], color=colors[i], label=name, lw=2.5)
ax1.set_title("DAX Index 1991-1998 Colored by Decoded HMM Regime (Viterbi Path)", fontsize=13,
↪ fontweight='bold')
ax1.set_ylabel("Index Level")
ax1.legend(loc='upper left')

# Bottom: raw sequence of decoded states over time
ax2.scatter(x, sorted_hidden_states, c=[colors[s] for s in sorted_hidden_states], s=8, marker='|')
ax2.set_yticks(range(n_components))
ax2.set_yticklabels(regime_names)
ax2.set_xlabel("Trading Day Index")

# Label the x-axis with years (real dates)
year_pos = np.where(dates.dt.year.diff().fillna(0).values != 0)[0]
ax2.set_xticks(year_pos)
ax2.set_xticklabels(dates.dt.year.iloc[year_pos].astype(str))

plt.tight_layout()
plt.show()

```

Experiment 17: Support Vector Machine (SVM)

Category: Supervised / margin-based learning

1. Objective

Train SVM classifiers and investigate kernels, C and gamma; also demonstrate Support Vector Regression with an ϵ -insensitive tube.

2. Datasets Used

- (a) Breast Cancer (569×30): two features (mean radius, mean texture) for the 2D kernel view and all 30 features for the accuracy benchmark. (b) Old Faithful geyser (data/geyser.csv, 272 eruptions) for SVR.

3. Required Libraries

scikit-learn, numpy, pandas, matplotlib

4. Brief Theory

SVC maximizes the margin while penalizing violations: $\min \frac{1}{2}||w||^2 + C \sum \xi_i$. The dual depends only on inner products $K(x_i, x_j)$ (kernel trick): linear, polynomial, RBF. C controls the violation penalty; γ controls the RBF influence radius. SVR ignores residuals within $\pm\epsilon$ and uses only outside points (support vectors).

5. Procedure

1. Scale features.
2. Train SVC with linear/poly/RBF kernels on the 2D cancer data; plot boundaries, margins and support vectors.
3. Train the RBF SVC on all 30 features; report accuracy and support-vector count.
4. Fit SVR (RBF, $C=10$, $\epsilon=0.3$) to waiting time $\rightarrow$ eruption duration; plot the fitted curve and ϵ -tube.
5. Report R^2 , RMSE and support-vector share.

6. Reference Implementation

```
clf = SVC(kernel=k_name, C=1.0, gamma='scale', degree=3, random_state=42).fit(X_2d_scaled, y_2d)
svc_rbf = SVC(kernel='rbf', C=1.0, random_state=42).fit(X_train_c_s, y_train_c)
svr_model = SVR(kernel='rbf', C=10.0, epsilon=0.3, gamma='scale').fit(X_svr_scaled, y_svr)
```

7. Results

Experiment	Metric	Value
SVC (RBF, 30 features)	Test accuracy	0.9790
	Support vectors	96 of 426 (22.5%)
SVR (Old Faithful)	R^2	0.8968
	RMSE	0.366 minutes
	Support vectors	103 of 272 (37.9%)

The 2D kernel figure shows the linear kernel underfitting the real diagnostic data while poly/RBF produce curved boundaries. The geyser scatter is bimodal; the RBF SVR smoothly bridges the two modes and the $\epsilon=0.3$ tube leaves most residuals unpenalized.

8. Discussion and Conclusion

SVMs give accurate, sparse solutions: 77% of the cancer training points could be discarded without changing the classifier. They require scaling and tuning (C , γ), and kernel methods scale as $O(n^2)$, which limits them on very large datasets.

9. Answers to Viva Questions

- **What is a support vector?** A training point on or inside the margin ($\alpha > 0$) that alone defines the decision boundary.
- **Effect of C ?** Large C penalizes violations strongly $\rightarrow$ narrow margin, possible overfitting; small C allows a wider margin and more violations.
- **Effect of γ ?** Large γ makes each point's influence local $\rightarrow$ wiggly boundary; small γ makes the RBF smoother, approaching a linear model.
- **Why does SVM require careful scaling?** Margins and kernels use Euclidean distances; unscaled features with larger ranges dominate them and degrade the fit.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/09_support_vector_machine`.
 $\hookrightarrow$ `ipynb` (executed; results above are its actual output).

Listing 17.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + model selection ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- SVM models: classification and regression ----
from sklearn.svm import SVC, SVR
# ---- Evaluation metrics ----
from sklearn.metrics import r2_score, root_mean_squared_error

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
 $\hookrightarrow$  default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("SVM modules loaded successfully!")
```

Listing 17.2 — 2D real data for visualizing margins and kernels

```
# ---- 2D real data for visualizing margins and kernels ----
# Two diagnostic measurements from the Breast Cancer dataset (malignant vs benign).
cancer_2d = load_breast_cancer()
X_2d = cancer_2d.data[:, [0, 1]] # mean radius and mean texture
y_2d = cancer_2d.target
feature_names_2d = ['mean radius', 'mean texture']

scaler_2d = StandardScaler()
X_2d_scaled = scaler_2d.fit_transform(X_2d)

kernels = ['linear', 'poly', 'rbf']
```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Coordinate grid used to draw the decision boundary as a contour
x_min, x_max = X_2d_scaled[:, 0].min() - 0.5, X_2d_scaled[:, 0].max() + 0.5
y_min, y_max = X_2d_scaled[:, 1].min() - 0.5, X_2d_scaled[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

for ax, k_name in zip(axes, kernels):
    # Train one SVC per kernel on the same real data
    clf = SVC(kernel=k_name, C=1.0, gamma='scale', degree=3, random_state=42)
    clf.fit(X_2d_scaled, y_2d)

    # Signed distance to the hyperplane for every grid point
    Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Filled contour = decision regions; solid line = boundary (Z=0); dashed = margins (Z=±1)
    ax.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 20), cmap='coolwarm', alpha=0.3)
    ax.contour(xx, yy, Z, levels=[-1.0, 0.0, 1.0], linestyles=['--', '-', '--'], colors=['k', 'k', 'k']
    ↪ ', linewidths=[1.5, 2.5, 1.5])

    # Scatter the data points (colored by class)
    ax.scatter(X_2d_scaled[:, 0], X_2d_scaled[:, 1], c=y_2d, cmap='coolwarm', edgecolors='k', s=25,
    ↪ alpha=0.8)

    # Highlight the support vectors (the points that define the margin)
    sv = clf.support_vectors_
    ax.scatter(sv[:, 0], sv[:, 1], s=90, facecolors='none', edgecolors='gold', linewidths=2.0, label=
    ↪ f'SV ({len(sv)})')

    ax.set_title(f"SVC with {k_name.capitalize()} Kernel\n(Acc: {clf.score(X_2d_scaled, y_2d):.3f},
    ↪ SVs: {len(sv)}", fontweight='bold')
    ax.set_xlabel(feature_names_2d[0])
    ax.set_ylabel(feature_names_2d[1])
    ax.legend(loc='lower right')

plt.suptitle("SVC Decision Boundaries on Real Breast Cancer Data (2 features)", fontsize=15,
    ↪ fontweight='bold')
plt.tight_layout()
plt.show()

```

Listing 17.3 — SVC on a real dataset: Breast Cancer (RBF kernel)

```

# ---- SVC on a real dataset: Breast Cancer (RBF kernel) ----
cancer = load_breast_cancer()
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    cancer.data, cancer.target, test_size=0.25, random_state=42, stratify=cancer.target
)
scaler_c = StandardScaler()
X_train_c_s = scaler_c.fit_transform(X_train_c)
X_test_c_s = scaler_c.transform(X_test_c)

svc_rbf = SVC(kernel='rbf', C=1.0, random_state=42)
svc_rbf.fit(X_train_c_s, y_train_c)
print(f"SVC (RBF) test accuracy on Breast Cancer: {svc_rbf.score(X_test_c_s, y_test_c):.4f}")
print(f"Support vectors used: {len(svc_rbf.support_)} of {len(X_train_c)} training samples")

```

Listing 17.4 — Real 1D regression: Old Faithful geyser

```

# ---- Real 1D regression: Old Faithful geyser ----
# x = waiting time before an eruption (minutes), y = eruption duration (minutes)
geyser = pd.read_csv('../data/geyser.csv')
X_svr = geyser['waiting'].values.reshape(-1, 1)
y_svr = geyser['eruptions'].values
print(f"Old Faithful: {len(geyser)} eruptions | waiting {X_svr.min()}-{X_svr.max()} min | duration {
    ↪ y_svr.min()}-{y_svr.max()} min")

```

```

# Scale the feature (SVR is distance-based); the target stays in minutes for interpretability
scaler_svr = StandardScaler()
X_svr_scaled = scaler_svr.fit_transform(X_svr)

# epsilon defines the width of the "no penalty" tube around the prediction (in minutes)
epsilon_val = 0.3
svr_model = SVR(kernel='rbf', C=10.0, epsilon=epsilon_val, gamma='scale')
svr_model.fit(X_svr_scaled, y_svr)

# Dense grid for drawing a smooth prediction curve, converted back to minutes
X_grid = np.linspace(X_svr_scaled.min(), X_svr_scaled.max(), 300).reshape(-1, 1)
y_grid_pred = svr_model.predict(X_grid)
X_grid_orig = scaler_svr.inverse_transform(X_grid).ravel()

# Support vectors = training points that lie on/outside the epsilon tube
sv_indices = svr_model.support_
X_sv = X_svr[sv_indices]
y_sv = y_svr[sv_indices]

plt.figure(figsize=(11, 6))
# Training samples
plt.scatter(X_svr, y_svr, color='navy', s=35, alpha=0.75, label='Eruptions')

# Regression surface
plt.plot(X_grid_orig, y_grid_pred, color='crimson', lw=2.5, label='SVR Prediction $f(x)$')

# Epsilon-insensitive tube boundaries + shaded tube
plt.plot(X_grid_orig, y_grid_pred + epsilon_val, color='gray', linestyle='--', lw=1.5, label=rf'Upper
↳ Tube ($+\epsilon$={epsilon_val}$)')
plt.plot(X_grid_orig, y_grid_pred - epsilon_val, color='gray', linestyle='--', lw=1.5, label=rf'Lower
↳ Tube ($-\epsilon$={epsilon_val}$)')
plt.fill_between(X_grid_orig, y_grid_pred - epsilon_val, y_grid_pred + epsilon_val, color='gray',
↳ alpha=0.15)

# Support vectors are highlighted - only these points influence the fitted curve
plt.scatter(X_sv, y_sv, s=120, facecolors='none', edgecolors='gold', linewidths=2.5, label=f'Support
↳ Vectors ({len(X_sv)})')

plt.title(f"Support Vector Regression on Old Faithful (C=10,  $\epsilon$ ={{epsilon_val}})", fontsize=14,
↳ fontweight='bold')
plt.xlabel("Waiting time before eruption (minutes)", fontsize=12)
plt.ylabel("Eruption duration (minutes)", fontsize=12)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

# ---- Quantitative evaluation ----
print(f"SVR R2 Score: {r2_score(y_svr, svr_model.predict(X_svr_scaled)):.4f}")
print(f"SVR RMSE:      {root_mean_squared_error(y_svr, svr_model.predict(X_svr_scaled)):.4f} minutes")
print(f"Support Vectors: {len(X_sv)} out of {len(X_svr)} points ({len(X_sv)/len(X_svr)*100:.1f}%)")

```

Experiment 18: Large Language Model (LLM) Experiment

Category: Generative / foundation models

1. Objective

Demonstrate a reproducible LLM workflow: tokenization, inference, controlled generation, and fine-tuning on a real public classification dataset.

2. Dataset Used

DistilBERT (SST-2) for tokenization/sentiment, DistilGPT2 for generation, and the SMS Spam Collection (data/sms_spam.csv, 5,572 labeled messages) for fine-tuning. A balanced subset (300 ham + 300 spam) is split 400 train / 200 stratified test.

3. Required Libraries

transformers, torch (CUDA), pandas

4. Brief Theory

LLMs process text as subword tokens (BPE/WordPiece) with an attention mask; self-attention computes $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T/\sqrt{d_k} + \text{mask})\mathbf{V}$. Causal generation predicts one token at a time; temperature scales logits and top-p restricts sampling to the nucleus. Fine-tuning reuses pretrained weights and replaces the classification head.

5. Procedure

1. Tokenize a sentence; inspect tokens, IDs and attention mask.
2. Run the sentiment pipeline on fixed test sentences.
3. Generate text with greedy decoding and with temperature/top-p sampling.
4. Load SMS Spam; build the balanced subset and stratified split.
5. Fine-tune DistilBERT (fresh 2-class head, AdamW lr 5e-5, 2 epochs, batch 16).
6. Evaluate accuracy/F1 on the untouched test messages and inspect examples.

6. Reference Implementation

```
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
    ↪ ignore_mismatched_sizes=True)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=5e-5)
outputs = ft_model(input_ids=ids, attention_mask=mask, labels=labels)
outputs.loss.backward(); optimizer.step()
```

7. Results

	()	
*	*	
0.50609	Value	
()	()	
*	*	
0.50609	Value	
0.705	Time	
loss	0.079	
(2		
epochs)		

```

() ()
* *
0.5060000000000000
0.5060000000000000
out
ac-
cu-
racy
() ()
* *
0.5060000000000000
out
F1
(spam)
() ()
* *
0.5060000000000000
am99.4%,
plesam
99.5%,
spam
89.7%
con-
fi-
dence
(all
cor-
rect)

```

Generation: greedy produced a safe deterministic continuation; nucleus sampling (T=0.7, p=0.9) produced a different but coherent sentence. Sentiment scores were in the 90-99% range on clear sentences.

8. Discussion and Conclusion

The experiment shows the complete practical LLM pipeline and that fine-tuning a small pre-trained model on 400 real messages already gives 96% spam detection. Limitations: the balanced subset ignores the natural 13% spam prior; the models are small and can hallucinate; fine-tuning data and prompts must be reproducible. No private data is used.

9. Answers to Viva Questions

- **What is tokenization?** Splitting text into subword units that map to integer vocabulary IDs, letting a fixed vocabulary represent any word; special tokens and attention masks handle sequence framing/padding.
 - **Pretraining vs fine-tuning vs prompting?** Pretraining learns general language representation from massive corpora; fine-tuning adapts those weights to a task with labeled data; prompting steers a frozen model through instructions/examples.
 - **Why is LLM output not automatically ground truth?** The model generates the most plausible continuation, not verified facts; it can hallucinate, and confidence scores are not calibrated correctness probabilities.
 - **How can prompt wording affect results?** Meaning, tone, format and label wording all change the conditional distribution the model samples from; small prompt changes can flip outputs, so prompts must be fixed and reported.
-

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/10_large_language_model.ipynb` (executed; results above are its actual output).

Listing 18.1 — Core numerical and plotting libraries

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- PyTorch: used for the fine-tuning demo ----
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
# ---- Dataset splitting + metrics for the fine-tune evaluation ----
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
# ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs ----
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForCausalLM,
    GenerationConfig,
    pipeline
)

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
↳ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

torch.manual_seed(42)
torch.cuda.manual_seed_all(42) # reproducible GPU training
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if available
print(f"PyTorch on {device}")
```

Listing 18.2 — Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based)

```
# ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
print(f"Loading Tokenizer: {model_id}...")
tokenizer = AutoTokenizer.from_pretrained(model_id)

sample_text = "Machine learning and deep neural networks are transforming modern science!"
# Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
encoded = tokenizer(
    sample_text,
    padding="max_length",
    max_length=20,
    truncation=True,
    return_tensors="pt"
)

# Convert integer IDs back to readable subword tokens
tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
print(f"Original Text: {sample_text}\n")
print(f"Subword Tokens ({len(tokens)}):\n", tokens)
print(f"Input IDs: \n", encoded['input_ids'][0].tolist())
print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())
```

Listing 18.3 — Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2)

```
# ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
classifier = pipeline("sentiment-analysis", model=model_id, device=0 if torch.cuda.is_available()
    ↪ else -1)

test_sentences = [
    "The new neural network model demonstrated extraordinary accuracy and remarkable speed.",
    "The dataset was poorly labeled, noisy, and the model completely failed to converge.",
    "The gradient descent algorithm performed reasonably well, meeting standard expectations.",
    "A catastrophic failure occurred during training due to exploding gradient issues."
]

# One forward pass per sentence -> label + calibrated confidence score
predictions = classifier(test_sentences)

# Collect results in a table for display
results_table = []
for sent, pred in zip(test_sentences, predictions):
    results_table.append({
        "Sentence": sent,
        "Predicted Label": pred['label'],
        "Confidence Score": f"{pred['score']*100:.2f}%"
    })

df_preds = pd.DataFrame(results_table)
display(df_preds)
```

Listing 18.4 — Load a small causal LM (DistilGPT2) for text generation

```
# ---- Load a small causal LM (DistilGPT2) for text generation ----
gen_model_id = "distilbert/distilgpt2"
print(f"Loading Text Generator: {gen_model_id}...")
# clean_up_tokenization_spaces=False keeps BPE spacing intact
generator = pipeline("text-generation", model=gen_model_id,
    device=0 if torch.cuda.is_available() else -1, clean_up_tokenization_spaces=
    ↪ False)

prompt = "Artificial Intelligence will fundamentally transform"

# 1. Greedy Search: always pick the most likely next token (deterministic)
greedy_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
)[0]['generated_text']

# 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up to cumulative p
sampled_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True, temperature=0.7,
    ↪ top_p=0.9)
)[0]['generated_text']

print(f"PROMPT: '{prompt}'\n")
print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---\n{sampled_output.strip()}")
```

Listing 18.5 — Real dataset: SMS Spam Collection (5,572 labeled messages)

```
# ---- Real dataset: SMS Spam Collection (5,572 labeled messages) ----
sms = pd.read_csv('../data/sms_spam.csv')
sms['y'] = (sms['label'] == 'spam').astype(int)
print(f"SMS Spam Collection: {len(sms)} messages | ham {(sms.y == 0).sum()}, spam {(sms.y == 1).sum()}
    ↪ ")

# Balanced subset for a fast fine-tune demo: 300 ham + 300 spam
ham = sms[sms.y == 0].sample(n=300, random_state=42)
```

```

spam = sms[sms.y == 1].sample(n=300, random_state=42)
subset = pd.concat([ham, spam]).sample(frac=1, random_state=42).reset_index(drop=True)

# Stratified split keeps the ham/spam ratio identical in train and test
train_df, test_df = train_test_split(subset, test_size=200, random_state=42, stratify=subset.y)
print(f"Train messages: {len(train_df)} | Test messages: {len(test_df)} | spam share: {subset.y.mean():.1%}")

class SMSDataset(Dataset):
    """Tokenize SMS texts and expose them as PyTorch samples."""
    def __init__(self, texts, labels, tokenizer):
        self.encodings = tokenizer(list(texts), padding=True, truncation=True, max_length=64,
        ↪ return_tensors="pt")
        self.labels = list(labels)

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)

train_loader = DataLoader(SMSDataset(train_df.text, train_df.y, tokenizer), batch_size=16, shuffle=
    ↪ True)
test_loader = DataLoader(SMSDataset(test_df.text, test_df.y, tokenizer), batch_size=32)

# ---- Load the pre-trained classifier with a fresh 2-class head ----
# ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
    ↪ ignore_mismatched_sizes=True)
ft_model.to(device)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=5e-5) # small LR for fine-tuning

print("Pre-trained Transformer loaded for fine-tuning!")

```

Listing 18.6 — Fine-tuning loop (2 epochs on 400 real SMS messages)

```

# ---- Fine-tuning loop (2 epochs on 400 real SMS messages) ----
epochs = 2
ft_model.train()
epoch_losses = []

for epoch in range(epochs):
    running_loss = 0.0
    for batch in train_loader:
        optimizer.zero_grad()

        # Move the batch to the selected device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Forward pass: the model computes the classification loss internally
        outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward() # backpropagate through the whole transformer
        optimizer.step() # update weights

        running_loss += loss.item() * len(input_ids) # de-average the batch loss

    avg_loss = running_loss / len(train_df)
    epoch_losses.append(avg_loss)
    print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

```

Listing 18.7 — Evaluate the fine-tuned model on held-out real SMS messages

```

# ---- Evaluate the fine-tuned model on held-out real SMS messages ----
ft_model.eval()
preds, targets = [], []

with torch.no_grad():
    for batch in test_loader:
        # Move the batch to the same device as the fine-tuned model (GPU when available)
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        logits = ft_model(input_ids=input_ids, attention_mask=attention_mask).logits
        preds += logits.argmax(-1).tolist()      # predicted class (0 = ham, 1 = spam)
        targets += batch['labels'].tolist()

print(f"Held-out test accuracy: {accuracy_score(targets, preds):.4f}")
print(f"Held-out test F1 (spam): {f1_score(targets, preds):.4f}")

# ---- Show a few individual predictions with confidence ----
labels_map = {0: "ham", 1: "spam"}
with torch.no_grad():
    for i in range(3):
        message = test_df.text.iloc[i]
        enc = tokenizer(message, truncation=True, max_length=64, return_tensors="pt").to(device)
        probs = torch.softmax(ft_model(**enc).logits, dim=1).cpu().numpy()[0]
        print(f"\nMessage: {message[:75]}...")
        print(f"True: {labels_map[test_df.y.iloc[i]]} | Predicted: {labels_map[probs.argmax()]} ({
↪ probs.max()*100:.1f}%)")

```

Experiment 19: Generalized Regression Neural Network (GRNN)

Category: Non-parametric regression

1. Objective

Implement GRNN-style kernel regression from scratch and study the effect of the smoothing parameter σ with cross-validation.

2. Dataset Used

Real motorcycle impact data (data/motorcycle.csv, R MASS::mcycle): 133 observations of time after impact (ms) versus head acceleration (g). Time is standardized before computing kernel distances.

3. Required Libraries

numpy, pandas, matplotlib, scikit-learn (API base classes, KFold, metrics)

4. Brief Theory

GRNN predicts the Nadaraya-Watson conditional mean: $\hat{y}(x) = \frac{\sum_i y_i \exp(-||x-x_i||^2/2\sigma^2)}{\sum_i \exp(-||x-x_i||^2/2\sigma^2)}$. Layer structure: input $\rightarrow$ pattern (one Gaussian per training point) $\rightarrow$ summation ($S = \sum y_i p_i$, $D = \sum p_i$) $\rightarrow$ output S/D . Training is one-pass (store the data); σ controls the bias-variance trade-off.

5. Procedure

1. Load mcycle; standardize time.
2. Implement the GRNN class (fit stores data; predict computes kernel weights).
3. Visualize $\sigma = 0.05 / 0.25 / 1.0$ (under/over-smoothing).
4. Select σ by 5-fold CV over 0.02...1.0.
5. Refit on a 75% split and evaluate on the 25% held-out set (R^2 , RMSE).

6. Reference Implementation

```
kernels = np.exp(-dist_sq / (2.0 * self.sigma ** 2)) # pattern layer
D = np.sum(kernels, axis=1) # denominator
S = np.dot(kernels, self.y_train_) # numerator
return S / np.where(D < 1e-12, 1e-12, D)
```

7. Results

Item	Value
CV-selected σ	0.080 (CV RMSE 25.00 g)
Train R^2	0.8073
Test R^2	0.7255
Test RMSE	22.78 g (acceleration range -134...+75 g)

$\sigma = 0.05$ shows local wiggles (variance), $\sigma = 0.25$ -1.0 underfits the sharp deceleration peak. The 5-fold CV curve has a clear minimum at $\sigma = 0.08$.

8. Discussion and Conclusion

GRNN is instant to train and needs only one hyperparameter. On the smooth 1D sine benchmark it reaches $R^2 \approx 0.95$, but on real motorcycle data the test R^2 is 0.73 — a single global σ cannot be simultaneously small at the impact discontinuity and large elsewhere. This is the classic bias-variance limitation of global-bandwidth kernel regression; adaptive bandwidths or local polynomial regression would improve it.

9. Answers to Viva Questions

- **Why is GRNN called non-parametric?** It does not learn a fixed set of parameters; the training data itself forms the model (memory-based kernel regression).
- **What does σ control?** The smoothing bandwidth: small σ fits noise (high variance), large σ over-smooths (high bias).
- **How is GRNN related to kernel regression?** It is exactly the Nadaraya-Watson Gaussian kernel regression estimator of $E[y | x]$, with one kernel per training observation.

10. Complete Code Listing

Full runnable program for this experiment, extracted from `notebooks/11_generalized_regression_neural_network` $\hookrightarrow$ `.ipynb` (executed; results above are its actual output).

Listing 19.1 — Core libraries

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, root_mean_squared_error

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else '
    ↪ default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 19.2 — GRNN class (from scratch)

```
class GRNN(BaseEstimator, RegressorMixin):
    """
    Generalized Regression Neural Network (Specht, 1991) - vectorized implementation.

    Prediction is the Nadaraya-Watson kernel regression estimate:
        y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
                  / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))

    Training is one-pass: the model simply stores the training set (no gradient descent).
    The only hyperparameter is the smoothing spread sigma.
    """

    def __init__(self, sigma=1.0):
        self.sigma = sigma

    def fit(self, X, y):
        # "Training" = memorizing the data (pattern + summation layers)
```

```

self.X_train_ = np.asarray(X, dtype=np.float64)
self.y_train_ = np.asarray(y, dtype=np.float64).ravel()
return self

def predict(self, X):
    X_test = np.asarray(X, dtype=np.float64)

    # ---- Pattern layer: squared Euclidean distance between each test and train point ----
    # Uses the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b for efficiency.
    dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
        np.sum(self.X_train_**2, axis=1, keepdims=True).T - \
        2 * np.dot(X_test, self.X_train_.T)
    dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values from rounding

    # ---- Pattern layer activation: Gaussian kernel of each training point ----
    kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))

    # ---- Summation layer: D-sum (denominator) and S-sum (numerator) ----
    D = np.sum(kernels, axis=1) # unweighted sum of activations
    S = np.dot(kernels, self.y_train_) # activation-weighted sum of targets

    # ---- Output layer: normalized weighted average (prevents 0/0 for isolated queries) ----
    D_safe = np.where(D < 1e-12, 1e-12, D)
    return S / D_safe

print("Custom vectorized GRNN class defined!")

```

Listing 19.3 — Load the real motorcycle impact data (MASS::mcycle)

```

# ---- Load the real motorcycle impact data (MASS::mcycle) ----
moto = pd.read_csv('../data/motorcycle.csv')
X = StandardScaler().fit_transform(moto['times'].values.reshape(-1, 1)) # time after impact (scaled)
y = moto['accel'].values # head acceleration (g)
print(f"Motorcycle data: {len(moto)} observations | time {moto.times.min()}-{moto.times.max()} ms |
↳ accel {moto.accel.min():.0f} to {moto.accel.max():.0f} g")

X_eval = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

# Under-smoothing, near-optimal, over-smoothing
sigmas = [0.05, 0.25, 1.0]
colors = ['purple', 'crimson', 'teal']

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, sig, col in zip(axes, sigmas, colors):
    # Fit a fresh GRNN for each sigma (training is instant)
    grnn_demo = GRNN(sigma=sig).fit(X, y)
    y_pred_demo = grnn_demo.predict(X_eval)

    ax.scatter(X, y, color='navy', s=25, alpha=0.6, label='Observations')
    ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=f'GRNN ({sig})')

    # Training R^2 shows the bias-variance behavior for each sigma
    r2 = r2_score(y, grnn_demo.predict(X))
    ax.set_title(f"Smoothing {sig} = {sig} (Train R^2 = {r2:.3f})", fontweight='bold')
    ax.set_xlabel("Time after impact (standardized)")
    ax.set_ylabel("Head acceleration (g)")
    ax.legend(loc='lower right')

plt.suptitle("GRNN Behavior on Real Motorcycle Impact Data σ (sweep)", fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

```

Listing 19.4 — 5-fold CV grid search for the optimal smoothing sigma

```
# ---- 5-fold CV grid search for the optimal smoothing sigma ----
sigma_candidates = np.linspace(0.02, 1.0, 50)
cv_scores = []
kf = KFold(n_splits=5, shuffle=True, random_state=42)  # fixed folds for a fair comparison

for sig in sigma_candidates:
    fold_rmse = []
    for train_idx, val_idx in kf.split(X):
        # Split the data into this fold's train/validation parts
        X_tr, y_tr = X[train_idx], y[train_idx]
        X_va, y_va = X[val_idx], y[val_idx]

        # Fit GRNN on the training fold and score on the validation fold
        model = GRNN(sigma=sig).fit(X_tr, y_tr)
        preds = model.predict(X_va)
        fold_rmse.append(root_mean_squared_error(y_va, preds))

    cv_scores.append(np.mean(fold_rmse))  # average CV error for this sigma

best_idx = np.argmin(cv_scores)
best_sigma = sigma_candidates[best_idx]
print(f"Optimal Smoothing Parameter  $\sigma$ : {best_sigma:.3f} (CV RMSE: {cv_scores[best_idx]:.4f} g)")

# ---- Plot the CV error curve and mark the optimum ----
plt.figure(figsize=(9, 4.5))
plt.plot(sigma_candidates, cv_scores, 'b-', lw=2.2)
plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal  $\sigma$  = {best_sigma:.3f}')
plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter  $\sigma$ ()", fontsize=13, fontweight='bold')
plt.xlabel("Smoothing Parameter  $\sigma$ ()", fontsize=11)
plt.ylabel("Cross-Validation RMSE (g)", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 19.5 — Final model: refit with the CV-selected sigma and evaluate on a held-out split

```
# ---- Final model: refit with the CV-selected sigma and evaluate on a held-out split ----
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42)
final_model = GRNN(sigma=best_sigma).fit(X_tr, y_tr)
y_hat = final_model.predict(X_te)

print(f"Final GRNN with CV-selected sigma = {best_sigma:.3f}")
print(f" - Train R2: {r2_score(y_tr, final_model.predict(X_tr)):.4f}")
print(f" - Test R2: {r2_score(y_te, y_hat):.4f}")
print(f" - Test RMSE: {root_mean_squared_error(y_te, y_hat):.4f} g")
```

```

() () () () ()
* * * * *
0.1667Model 1: 167 Observation
K=5 Test ROC
H=1667 AUC
peru-
0.1667 accuracy
ram-
e-
0.1667 ters

```

```

() () () () ()
* * * * *
0.1667Model 2: 167 sep-
(RF) (Brakst a-
Can- rate
cer) 30-
feature
bench-
mark

```

Regression comparison — California Housing (5,000 sample, same 80/20 split)

```

() () () () ()
* * * * *
0.2000Model 1: 167 Observation
0.2000R^2

```

```

() () () () ()
* * * * *
0.2000Model 2: 167 Observation
dom 0.7513
For- ≈
est test,
Re- no
gres- over-
sor fit
() () () () ()
* * * * *
0.2000Model 3: 167 Observation
Boost trade-
Re- off
gres-
sor
() () () () ()
* * * * *
0.2000Model 4: 167 Observation
aBoost sensitive
Re-
gres-
sor
() () () () ()
* * * * *
0.2000Model 5: 167 Observation
Boost XG-
Re- Boost
gres-
sor

```

Clustering comparison — Mall Customers (K=5, standardized)

data; density methods are the only option for arbitrary shapes and noise. (4) Fair comparison requires identical splits, fixed seeds, untouched test sets, and reporting at least two metrics per task.

Common mistakes avoided. Scaling fitted only on training data; no repeated test-set peeking; same partitions for all models; F1/AUC reported alongside accuracy; cluster labels never treated as ground truth; one hyperparameter changed at a time; prompts and model versions recorded; no private data sent to external models.

Appendix A: Quick Reference Formulas

Concept	Formula / Definition
Euclidean distance	$d(x,y) = \sqrt{\sum_j (x_j - y_j)^2}$
K-means objective	$J = \sum_i$
Regression MAE	$MAE = (1/n) \sum_i y_i - \hat{y}_i $
Regression RMSE	$RMSE = \sqrt{(1/n) \sum_i (y_i - \hat{y}_i)^2}$
R^2	$R^2 = 1 - SS_{res} / SS_{tot}$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
SVM margin idea	Maximize margin while penalizing violations via C
GRNN kernel	$w_i(x) = \exp(- x - x_i ^2 / (2\sigma^2))$

Appendix B: Reproducibility Checklist

- Python 3.12 (.venv) and all package versions recorded (pip [list](#) in the repository documentation).
- Random seeds fixed everywhere (random_state=42, random_seed=42, torch.manual_seed(42), torch.cuda.manual_seed_all(42)).
- Datasets committed under data/ with documented sources (USGS, UCI, scikit-learn, Rdatasets, SMS Spam Collection).
- Preprocessing recorded: scaling fitted on training data only; haversine radians for geospatial data; MinMax for SOM/RNN.
- Train/validation/test protocols recorded (stratified, chronological, early-stopping split, 5-fold CV).
- Hyperparameters documented per experiment in the notebooks and this report.
- Metrics defined before final testing; no test-set model selection.
- Figures and tables saved inside the executed notebooks and the printed PDFs.
- Code runs from the committed .venv with the listed dependencies; GPU optional (CPU fallback automatic).

Appendix C: Suggested Assessment Rubric

Component	Weight
Pre-lab preparation / algorithm understanding	10%
Correct implementation	25%
Experimental design and preprocessing	15%
Evaluation and visualization	15%
Analysis and interpretation	20%
Report quality / reproducibility	10%
Viva / discussion	5%

Appendix D: Student Experiment Record

Field	Student Entry
Name	
ID	
Section	
Experiment No.	
Date	
Dataset	
Algorithm / Version	
Key Hyperparameters	
Random Seed	
Main Result	
Observation	
Instructor Signature	